XI'AN JIAOTONG-LIVERPOOL UNIVERSITY
西 交 利 物 浦 大 学

COURSEWORK SUBMISSION COVER SHEET

Name	(Surname)	(Other Names)
Student Number	Yu Lu 2364294	
Programme	Biomedical Statistics	
Module Title	Nummerical Analysis	
Module Code	MTH208	
Assignment Title	Coursework II	
Submission Deadline	May 31th	
Module Leader	Qiang Niu	

By uploading or submitting this coursework submission cover sheet, I certify the following:

- I have read and understood the definitions of collusion, copying, plagiarism, dishonest use of data, and academic offences involving Artificial Intelligence (AI) as outlined in the Academic Integrity Policy of Xi'an Jiaotong-Liverpool University.
- This work is my own, original work produced specifically for this assignment. It does not misrepresent the work of another person or institution, nor does it present the work generated by AI as my own. Additionally, it is a submission that has not been previously published, or submitted to another module.
- This work is not the product of unauthorized collaboration between myself and others.
- This work is free of embellished or fabricated data.

I understand that collusion, copying, plagiarism, dishonest use of data, academic offences involving AI, and submitting procured work are serious academic misconduct. By uploading or submitting this cover sheet, I acknowledge that I am subject to disciplinary action if I am found to have committed such acts.

Signature ..... Date ..... May 31th

For Academic Office use:	Date Received	Working Days Late	Penalty[1]

[1] Please refer to clause 43 (i), (ii) and (iii) of the Code of Practice on Assessment for the standard system of penalties for the late submission of assessment work based on **working days**.

Numerical Analysis Coursework 2: Function Approximation, Orthogonal Polynomials, Vandermonde Systems, and the Arnoldi Method

May 31, 2026

Abstract

This report studies polynomial interpolation and least-squares approximation, with emphasis on numerical conditioning and stability. A classical counterexample is used to demonstrate the instability of equispaced interpolation and the improvement obtained by Chebyshev node clustering. For the least-squares approximation of smooth functions, three bases are compared: monomials, Chebyshev polynomials, and an Arnoldi-generated discrete orthonormal basis. The numerical results show that monomial Vandermonde systems become severely ill-conditioned as the polynomial degree increases, whereas Chebyshev and Arnoldi bases maintain excellent stability. Finally, Legendre and regularised Chebyshev approximations are compared for functions with endpoint singularities. The experiments confirm that node distribution, basis selection, and the choice of linear algebra methods are all essential components for achieving stable and accurate numerical computation.

Contents

1 Introduction **2**

2 Methodology and Algorithms **3**
2.1 Vandermonde interpolation and conditioning (Task 1(a), Task 1(b)) 3
2.1.1 Determinant of the Vandermonde matrix (Task 1(a)) 3
2.1.2 Equispaced nodes and growth of the condition number (Task 1(b)) 5
2.2 Interpolation for the Runge experiment (Task 1(d), Task 2(a)–(d)) 11
2.2.1 Lebesgue constants and the Runge phenomenon (Task 1(d)) 11
2.3 Chebyshev nodes and Chebyshev basis (Task 1(c), Task 3(c), Task 4(b)) 13
2.3.1 Chebyshev recurrence and orthogonality (Task 1(c)) 13
2.4 Least-squares approximation (Task 3(a), Task 3(b)) 13
2.4.1 Monomial basis with normal equations (Task 3(b)) 13
2.5 QR method (Task 3(c)) 14
2.5.1 Chebyshev basis with QR decomposition (Task 3(c)) 14
2.5.2 Why the Chebyshev–QR method is more stable (Task 3(c)) 15
2.6 Arnoldi orthonormal polynomial basis (Task 3(d)) 17
2.6.1 Arnoldi-generated orthonormal basis by Gram–Schmidt (Task 3(d)) . . . 17
2.7 Gauss–Legendre and Chebyshev–Gauss quadrature (Task 4(a), Task 4(b)) 19

3 Numerical Results and Analysis **20**
3.1 Task 2: Runge phenomenon (Task 2(a)–(d)) 20
3.2 Task 3: Monomial vs Chebyshev vs Arnoldi least squares (Task 3(a)–(e)) 24
3.2.1 Arnoldi coefficient decay (Task 3(d)) 24
3.2.2 Error comparison (Task 3(e)) 25

3.2.3 Why the condition numbers differ (Task 3(e)) . . . 25
3.2.4 Computational cost (Task 3(e)) . . . 29
3.3 Task 4: Legendre vs regularised Chebyshev approximation (Task 4(a)–(d)) . . . 29
3.3.1 Legendre approximation (Task 4(a)) . . . 29
3.3.2 Regularised Chebyshev approximation (Task 4(b)) . . . 29
3.3.3 Legendre expansion (Task 4(a)) . . . 30
3.3.4 Regularised Chebyshev expansion (Task 4(b)) . . . 31
3.3.5 Convergence comparison (Task 4(c)) . . . 33
3.3.6 When Legendre approximation is preferable (Task 4(d)) . . . 35
3.3.7 Connection between Arnoldi and Gaussian quadrature (Task 4(d)) . . . 35

4 Concluding Remarks **35**

5 References **36**

A MATLAB Code **37**

1 Introduction

Function approximation is a fundamental problem in numerical analysis. Polynomial interpolation and least-squares approximation are especially important because they provide simple and flexible ways to approximate functions from sampled data [1]. However, the accuracy of these methods is not determined only by the polynomial degree. It also depends strongly on the distribution of interpolation nodes, the choice of polynomial basis, and the numerical linear algebra method used to compute the coefficients [2, 3].

A central motivation of this report is to investigate why polynomial approximation can become numerically unstable. In interpolation, the monomial Vandermonde matrix is often ill-conditioned, especially when the nodes are equispaced. This ill-conditioning amplifies rounding errors and can produce large oscillations near the endpoints of the interval. The classical example is the Runge phenomenon, where interpolation of a smooth-looking function on equispaced nodes fails as the degree increases [2]. Chebyshev nodes reduce this effect by clustering near the endpoints and by keeping the Lebesgue constant much smaller than in the equispaced case [2, 4].

Another aim is to compare different bases for polynomial least-squares approximation. Although the monomial, Chebyshev, Legendre, and Arnoldi-generated bases may span the same polynomial space in exact arithmetic, they can behave very differently in floating-point arithmetic. A monomial Vandermonde matrix may become severely ill-conditioned, whereas Chebyshev and other orthogonal polynomial bases usually produce much better conditioned approximation problems [5, 3]. This report therefore compares monomial normal-equation least squares with QR-based Chebyshev least squares, since QR factorisation avoids the squaring of the condition number that occurs in the normal equations [6, 5].

The Arnoldi method is also studied as a way of constructing a discrete orthonormal polynomial basis from sampled data. In numerical linear algebra, Arnoldi iteration is a Krylov subspace method for generating orthonormal bases and Hessenberg recurrences [6, 7]. When applied to the multiplication-by-x operator on a discrete set of nodes, it produces an orthonormal polynomial basis adapted to the sampling grid. This provides a stable alternative to explicitly forming high-degree monomials and gives a direct connection between polynomial approximation, Krylov subspaces, and recurrence relations for orthogonal polynomials [7, 3].

Finally, this report studies weighted approximation using Legendre and Chebyshev polynomial expansions. Legendre polynomials are orthogonal with respect to the uniform weight on $[-1, 1]$, while Chebyshev polynomials are orthogonal with respect to the singular weight $(1-x^2)^{-1/2}$ [**?**, 3]. This distinction is important for functions with endpoint singularities. If the weight structure

matches the singular behaviour of the function, the approximation may converge much faster. This motivates the comparison between a Legendre expansion and a regularised Chebyshev expansion for an endpoint-singular test function [4, 2].

Overall, the report investigates three linked questions: how node distributions affect interpolation stability, how polynomial bases affect least-squares conditioning, and how orthogonal-polynomial structure can be used to obtain more stable and accurate approximations. The numerical experiments are designed to demonstrate these issues through the Runge phenomenon, monomial versus orthogonal-basis least squares, Arnoldi orthonormalisation, and Legendre–Chebyshev approximation for functions with endpoint singularities.

2 Methodology and Algorithms

2.1 Vandermonde interpolation and conditioning (Task 1(a), Task 1(b))

2.1.1 Determinant of the Vandermonde matrix (Task 1(a))

Let

$$V_n(x_0, x_1, \ldots, x_n) = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix}.$$

We prove by mathematical induction that

$$\det(V_n) = \prod_{0 \le i < j \le n} (x_j - x_i).$$

Base case: For $n = 1$, the matrix is

$$V_1(x_0, x_1) = \begin{pmatrix} 1 & x_0 \\ 1 & x_1 \end{pmatrix}.$$

Hence

$$\det(V_1) = x_1 - x_0.$$

This agrees with

$$\prod_{0 \le i < j \le 1} (x_j - x_i) = x_1 - x_0.$$

Thus the formula holds for $n = 1$.

Hypothesis: Assume that the formula holds for V_{n-1}. That is, for any n nodes $y_0, y_1, \ldots, y_{n-1}$,

$$\det(V_{n-1}(y_0, y_1, \ldots, y_{n-1})) = \prod_{0 \le i < j \le n-1} (y_j - y_i).$$

Inductive step: We prove the formula for

$$V_n(x_0, x_1, \ldots, x_n),$$

which is an $(n+1) \times (n+1)$ Vandermonde matrix.

Case 1: If two nodes are equal, then two rows of V_n are identical, so

$$\det(V_n) = 0.$$

Meanwhile, the product

$$\prod_{0 \le i < j \le n} (x_j - x_i)$$

also contains a zero factor. Therefore the formula holds in this case. Hence, it remains to consider the case where all nodes are distinct.

Case 2.1: Suppose all nodes are distinct and one of the nodes is zero.
By relabelling the nodes if necessary, we may assume

$$x_0 = 0.$$

Then the first row of V_n is

$$(1, 0, 0, \ldots, 0).$$

Expanding the determinant along the first row gives

$$\det(V_n(0, x_1, \ldots, x_n)) = \det \begin{pmatrix} x_1 & x_1^2 & \cdots & x_1^n \\ x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & & \vdots \\ x_n & x_n^2 & \cdots & x_n^n \end{pmatrix}.$$

Factoring x_i from the row corresponding to each x_i, for $i = 1, \ldots, n$, yields

$$\det(V_n(0, x_1, \ldots, x_n)) = \left(\prod_{j=1}^{n} x_j \right) \det(V_{n-1}(x_1, x_2, \ldots, x_n)).$$

By the induction hypothesis,

$$\det(V_{n-1}(x_1, x_2, \ldots, x_n)) = \prod_{1 \le i < j \le n} (x_j - x_i).$$

Hence

$$\det(V_n(0, x_1, \ldots, x_n)) = \left(\prod_{j=1}^{n} x_j \right) \prod_{1 \le i < j \le n} (x_j - x_i).$$

Since $x_j = x_j - 0 = x_j - x_0$, we have

$$\prod_{j=1}^{n} x_j = \prod_{j=1}^{n} (x_j - x_0).$$

Therefore

$$\det(V_n(0, x_1, \ldots, x_n)) = \prod_{j=1}^{n} (x_j - x_0) \prod_{1 \le i < j \le n} (x_j - x_i) = \prod_{0 \le i < j \le n} (x_j - x_i).$$

Thus the formula holds when one of the nodes is zero.

Case 2.2 Suppose all nodes are distinct and nonzero. Fix $x_1, \ldots, x_n$, and replace x_0 by a variable x. Define

$$p(x) = \det(V_n(x, x_1, \ldots, x_n)).$$

Since only the first row depends on x, $p(x)$ is a polynomial in x of degree at most n. For $i = 1, \ldots, n$, if $x = x_i$, then two rows of the matrix are identical. Therefore

$$p(x_i) = 0, \qquad i = 1, \ldots, n.$$

Hence $x_1, \ldots, x_n$ are roots of $p(x)$.
Since $p(x)$ has degree at most n, it follows that

$$p(x) = C \prod_{j=1}^{n} (x_j - x),$$

where C is independent of x.
To find C, set $x = 0$. By Case 2.1,

$$p(0) = \det(V_n(0, x_1, \ldots, x_n)) = \left(\prod_{j=1}^{n} x_j\right) \prod_{1 \le i < j \le n} (x_j - x_i).$$

On the other hand,

$$p(0) = C \prod_{j=1}^{n} x_j.$$

Since all $x_j \neq 0$, we may cancel

$$\prod_{j=1}^{n} x_j.$$

Hence

$$C = \prod_{1 \le i < j \le n} (x_j - x_i).$$

Therefore

$$p(x) = \prod_{1 \le i < j \le n} (x_j - x_i) \prod_{j=1}^{n} (x_j - x).$$

Substituting $x = x_0$, we obtain

$$\det(V_n(x_0, x_1, \ldots, x_n)) = \prod_{1 \le i < j \le n} (x_j - x_i) \prod_{j=1}^{n} (x_j - x_0).$$

Combining the factors gives

$$\det(V_n(x_0, x_1, \ldots, x_n)) = \prod_{0 \le i < j \le n} (x_j - x_i).$$

Conclusion: Therefore, by mathematical induction,

$$\det(V_n) = \prod_{0 \le i < j \le n} (x_j - x_i)$$

.

2.1.2 Equispaced nodes and growth of the condition number (Task 1(b))

Let the equispaced nodes on $[-1, 1]$ be

$$x_i = -1 + \frac{2i}{n}, \qquad i = 0, 1, \ldots, n.$$

The corresponding Vandermonde matrix is

$$V = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix}.$$

Throughout this proof, $\|\cdot\|$ denotes the matrix 2-norm:

$$\|A\| = \max_{z \neq 0} \frac{\|Az\|_2}{\|z\|_2}.$$

The condition number is

$$\kappa(V) = \|V\| \|V^{-1}\|.$$

Connection between the condition number and singular values. Let the singular value decomposition of V be

$$V = U\Sigma W^T,$$

where U and W are orthogonal matrices and

$$\Sigma = \operatorname{diag}(\sigma_0, \sigma_1, \ldots, \sigma_n),$$

with

$$\sigma_0 \geq \sigma_1 \geq \cdots \geq \sigma_n > 0.$$

Since U and W are orthogonal matrices, they preserve the Euclidean norm. For any vector a,

$$\|Ua\|_2^2 = (Ua)^T(Ua) = a^T U^T U a = a^T a = \|a\|_2^2.$$

Hence

$$\|Ua\|_2 = \|a\|_2.$$

The same argument applies to W and W^T.

For any vector z,

$$\|Vz\|_2 = \|U\Sigma W^T z\|_2 = \|\Sigma W^T z\|_2.$$

Let

$$y = W^T z.$$

Since W is orthogonal,

$$\|y\|_2 = \|z\|_2.$$

Therefore

$$\|V\| = \max_{z \neq 0} \frac{\|Vz\|_2}{\|z\|_2} = \max_{y \neq 0} \frac{\|\Sigma y\|_2}{\|y\|_2}.$$

Now

$$\|\Sigma y\|_2^2 = \sum_{j=0}^{n} \sigma_j^2 y_j^2.$$

Since σ_0 is the largest singular value,

$$\sum_{j=0}^{n} \sigma_j^2 y_j^2 \leq \sigma_0^2 \sum_{j=0}^{n} y_j^2 = \sigma_0^2 \|y\|_2^2.$$

Thus

$$\frac{\|\Sigma y\|_2}{\|y\|_2} \leq \sigma_0.$$

Equality is obtained by taking

$$y = (1, 0, \ldots, 0)^T.$$

Therefore

$$\|V\| = \sigma_0.$$

Similarly,

$$V^{-1} = W\Sigma^{-1}U^T,$$

where

$$\Sigma^{-1} = \operatorname{diag}\left(\frac{1}{\sigma_0}, \frac{1}{\sigma_1}, \ldots, \frac{1}{\sigma_n}\right).$$

The largest diagonal entry of Σ^{-1} is $1/\sigma_n$. Hence

$$\|V^{-1}\| = \frac{1}{\sigma_n}.$$

Therefore

$$\kappa(V) = \|V\|\|V^{-1}\| = \sigma_0 \cdot \frac{1}{\sigma_n} = \frac{\sigma_0}{\sigma_n}.$$

Determinant of the equispaced Vandermonde matrix. For equispaced nodes,

$$x_j - x_i = \left(-1 + \frac{2j}{n}\right) - \left(-1 + \frac{2i}{n}\right) = \frac{2(j-i)}{n}.$$

Using the Vandermonde determinant formula,

$$\det(V) = \prod_{0 \le i < j \le n} (x_j - x_i),$$

we get

$$|\det(V)| = \prod_{0 \le i < j \le n} \frac{2(j-i)}{n}.$$

There are

$$\binom{n+1}{2} = \frac{n(n+1)}{2}$$

pairs (i, j) satisfying $0 \le i < j \le n$. Hence

$$|\det(V)| = \left(\frac{2}{n}\right)^{n(n+1)/2} \prod_{0 \le i < j \le n} (j-i).$$

For fixed j,

$$\prod_{i=0}^{j-1} (j-i) = j(j-1)\cdots 1 = j!.$$

Therefore

$$\prod_{0 \le i < j \le n} (j-i) = \prod_{j=1}^{n} j!.$$

So

$$|\det(V)| = \left(\frac{2}{n}\right)^{n(n+1)/2} \prod_{j=1}^{n} j!.$$

Estimating the determinant using Stirling's formula. Taking logarithms gives

$$\log|\det(V)| = \frac{n(n+1)}{2} \log\left(\frac{2}{n}\right) + \sum_{j=1}^{n} \log(j!).$$

We now estimate the second term using Stirling's formula:

$$j! = \sqrt{2\pi j}\left(\frac{j}{e}\right)^j \left(1 + O\left(\frac{1}{j}\right)\right).$$

Taking logarithms,

$$\log(j!) = \log\left[\sqrt{2\pi j}\left(\frac{j}{e}\right)^j \left(1 + O\left(\frac{1}{j}\right)\right)\right].$$

Using $\log(ab) = \log a + \log b$,

$$\log(j!) = \log\left(\sqrt{2\pi j}\right) + \log\left(\left(\frac{j}{e}\right)^j\right) + \log\left(1 + O\left(\frac{1}{j}\right)\right).$$

Now

$$\log\left(\sqrt{2\pi j}\right) = \frac{1}{2}\log(2\pi j),$$

and

$$\log\left(\left(\frac{j}{e}\right)^j\right) = j\log\left(\frac{j}{e}\right) = j(\log j - 1) = j\log j - j.$$

Also,

$$\log\left(1 + O\left(\frac{1}{j}\right)\right) = O\left(\frac{1}{j}\right).$$

Hence

$$\log(j!) = j\log j - j + \frac{1}{2}\log(2\pi j) + O\left(\frac{1}{j}\right).$$

For the leading asymptotic estimate, this can be written as

$$\log(j!) = j\log j - j + O(\log j).$$

Therefore

$$\sum_{j=1}^{n}\log(j!) = \sum_{j=1}^{n} j\log j - \sum_{j=1}^{n} j + O\left(\sum_{j=1}^{n}\log j\right).$$

Since $j \le n$ implies $\log j \le \log n$,

$$\sum_{j=1}^{n}\log j \le n\log n.$$

Hence

$$\sum_{j=1}^{n}\log j = O(n\log n).$$

Thus

$$\sum_{j=1}^{n}\log(j!) = \sum_{j=1}^{n} j\log j - \sum_{j=1}^{n} j + O(n\log n).$$

Now

$$\sum_{j=1}^{n} j = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + O(n).$$

To estimate

$$\sum_{j=1}^{n} j\log j,$$

we compare it with the integral of $x\log x$. Since

$$\int x\log x\,dx = \frac{x^2}{2}\log x - \frac{x^2}{4},$$

we have

$$\int_1^n x\log x\,dx = \left[\frac{x^2}{2}\log x - \frac{x^2}{4}\right]_1^n = \frac{1}{2}n^2\log n - \frac{1}{4}n^2 + O(1).$$

The difference between the sum and this integral is of order $O(n\log n)$, so

$$\sum_{j=1}^{n} j\log j = \frac{1}{2}n^2\log n - \frac{1}{4}n^2 + O(n\log n).$$

Therefore

$$\sum_{j=1}^{n}\log(j!) = \left(\frac{1}{2}n^2\log n - \frac{1}{4}n^2\right) - \frac{1}{2}n^2 + O(n\log n).$$

Thus

$$\sum_{j=1}^{n} \log(j!) = \frac{1}{2} n^2 \log n - \frac{3}{4} n^2 + O(n \log n).$$

Next,

$$\frac{n(n+1)}{2} \log\left(\frac{2}{n}\right) = \frac{n(n+1)}{2} (\log 2 - \log n).$$

Since

$$\frac{n(n+1)}{2} = \frac{1}{2} n^2 + O(n),$$

we obtain

$$\frac{n(n+1)}{2} \log\left(\frac{2}{n}\right) = \frac{1}{2} n^2 \log 2 - \frac{1}{2} n^2 \log n + O(n \log n).$$

Substituting the two estimates into $\log |\det(V)|$, we get

$$\log |\det(V)| = \left(\frac{1}{2} n^2 \log 2 - \frac{1}{2} n^2 \log n\right) + \left(\frac{1}{2} n^2 \log n - \frac{3}{4} n^2\right) + O(n \log n).$$

The $n^2 \log n$-terms cancel. Hence

$$\log |\det(V)| = \left(\frac{1}{2} \log 2 - \frac{3}{4}\right) n^2 + O(n \log n).$$

Define

$$c_0 = \frac{3}{4} - \frac{1}{2} \log 2 > 0.$$

Then

$$\log |\det(V)| = -c_0 n^2 + O(n \log n).$$

By the definition of the $O(n \log n)$ term, there exists a constant $M > 0$ such that, for sufficiently large n,

$$|O(n \log n)| \leq M n \log n.$$

Since n^2 grows faster than $n \log n$, we have

$$M n \log n \leq \frac{c_0}{2} n^2$$

for sufficiently large n. Therefore

$$\log |\det(V)| \leq -c_0 n^2 + \frac{c_0}{2} n^2 = -\frac{c_0}{2} n^2.$$

Let

$$c = \frac{c_0}{2} > 0.$$

Then, for sufficiently large n,

$$|\det(V)| \leq e^{-cn^2}.$$

From a small determinant to a large condition number. Using the SVD,

$$V = U\Sigma W^T,$$

we have

$$|\det(V)| = |\det(U)\det(\Sigma)\det(W^T)|.$$

Since U and W are orthogonal matrices,

$$|\det(U)| = |\det(W^T)| = 1.$$

Therefore

$$|\det(V)| = |\det(\Sigma)| = \prod_{j=0}^{n} \sigma_j.$$

Because σ_n is the smallest singular value,

$$\sigma_j \geq \sigma_n, \qquad j = 0, 1, \ldots, n.$$

Thus

$$|\det(V)| = \prod_{j=0}^{n} \sigma_j \geq \sigma_n^{n+1}.$$

Hence

$$\sigma_n \leq |\det(V)|^{1/(n+1)}.$$

Using

$$|\det(V)| \leq e^{-cn^2},$$

we get

$$\sigma_n \leq e^{-cn^2/(n+1)}.$$

For $n \geq 1$,

$$\frac{n^2}{n+1} \geq \frac{n}{2}.$$

Thus

$$\sigma_n \leq e^{-cn/2}.$$

Let

$$c_1 = \frac{c}{2} > 0.$$

Then

$$\sigma_n \leq e^{-c_1 n}.$$

On the other hand, the largest singular value is not small. Let

$$e_0 = (1, 0, \ldots, 0)^T.$$

Since the first column of V is the vector of all ones,

$$Ve_0 = (1, 1, \ldots, 1)^T.$$

Therefore

$$\|Ve_0\|_2 = \sqrt{n+1}, \qquad \|e_0\|_2 = 1.$$

Since

$$\sigma_0 = \|V\| = \max_{\|z\|_2=1} \|Vz\|_2,$$

we have

$$\sigma_0 \geq \sqrt{n+1}.$$

Therefore

$$\kappa(V) = \frac{\sigma_0}{\sigma_n} \geq \frac{\sqrt{n+1}}{e^{-c_1 n}} = \sqrt{n+1}\, e^{c_1 n}.$$

This proves that the condition number grows at least exponentially with n.

2.2 Interpolation for the Runge experiment (Task 1(d), Task 2(a)–(d))

2.2.1 Lebesgue constants and the Runge phenomenon (Task 1(d))

Let $x_0, x_1, \ldots, x_n$ be distinct interpolation nodes in $[-1, 1]$. The Lagrange basis polynomials are

$$\ell_i(x) = \prod_{\substack{0 \le j \le n \\ j \ne i}} \frac{x - x_j}{x_i - x_j}, \qquad i = 0, 1, \ldots, n,$$

and the interpolation operator is

$$I_n f(x) = \sum_{i=0}^{n} f(x_i)\ell_i(x).$$

The Lebesgue function and Lebesgue constant are defined by

$$\lambda_n(x) = \sum_{i=0}^{n} |\ell_i(x)|, \qquad \Lambda_n = \max_{x \in [-1,1]} \lambda_n(x).$$

Stability of the interpolation operator. For any continuous function g on $[-1, 1]$,

$$|I_n g(x)| = \left| \sum_{i=0}^{n} g(x_i)\ell_i(x) \right| \le \sum_{i=0}^{n} |g(x_i)|\,|\ell_i(x)|.$$

Since

$$|g(x_i)| \le \max_{x \in [-1,1]} |g(x)|, \qquad i = 0, 1, \ldots, n,$$

it follows that

$$|I_n g(x)| \le \left(\max_{x \in [-1,1]} |g(x)| \right) \sum_{i=0}^{n} |\ell_i(x)| = \left(\max_{x \in [-1,1]} |g(x)| \right) \lambda_n(x).$$

Taking the maximum over $x \in [-1, 1]$ gives

$$\max_{x \in [-1,1]} |I_n g(x)| \le \Lambda_n \max_{x \in [-1,1]} |g(x)|.$$

Thus the Lebesgue constant is the stability factor of the interpolation operator.

Let $p^* \in \mathcal{P}_n$ be a best polynomial approximation to f on $[-1, 1]$, namely

$$\max_{x \in [-1,1]} |f(x) - p^*(x)| = \min_{p \in \mathcal{P}_n} \max_{x \in [-1,1]} |f(x) - p(x)|.$$

Since $I_n p^* = p^*$, we have

$$f(x) - I_n f(x) = \big(f(x) - p^*(x)\big) - I_n\big(f - p^*\big)(x).$$

Therefore

$$|f(x) - I_n f(x)| \le |f(x) - p^*(x)| + \left|I_n\big(f - p^*\big)(x)\right|.$$

Taking the maximum over $x \in [-1, 1]$, and using the stability estimate with $g = f - p^*$, gives

$$\max_{x \in [-1,1]} |f(x) - I_n f(x)| \le (1 + \Lambda_n) \max_{x \in [-1,1]} |f(x) - p^*(x)|.$$

By the definition of p^*,

$$\max_{x \in [-1,1]} |f(x) - I_n f(x)| \le (1 + \Lambda_n) \min_{p \in \mathcal{P}_n} \max_{x \in [-1,1]} |f(x) - p(x)|.$$

This proves that interpolation is accurate only when the best approximation error is small and the Lebesgue constant is not too large.

Equispaced nodes. Let the equispaced interpolation nodes be

$$x_j = -1 + \frac{2j}{n}, \qquad j = 0, 1, \ldots, n.$$

Denote the corresponding Lebesgue constant by Λ_n^{eq}. The Lebesgue constant for polynomial interpolation at equidistant nodes has the asymptotic form

$$\Lambda_n^{\text{eq}} \sim \frac{2^{n+1}}{e\, n \log n}, \qquad n \to \infty,$$

as stated for equidistant-node interpolation in the literature [8]. This means that

$$\lim_{n\to\infty} \frac{\Lambda_n^{\text{eq}}}{2^{n+1}/(e\, n \log n)} = 1.$$

Consequently, for all sufficiently large n,

$$\Lambda_n^{\text{eq}} \geq \frac{1}{2} \frac{2^{n+1}}{e\, n \log n}.$$

Since $2^n/(n \log n)$ grows exponentially relative to any power of $\log n$, the interpolation operator associated with equispaced nodes becomes rapidly ill-conditioned as n increases.

A lower-bound proof for equally spaced nodes is also given by Rivlin. For a symmetric equidistant set with $2m+1$ nodes and degree $2m$, the Lebesgue constant satisfies a lower bound of the form

$$\Lambda_{2m}^{\text{eq}} \geq C \frac{4^m}{m^\alpha},$$

where $C > 0$ and $\alpha > 0$ are constants independent of m [9]. Since $4^m = 2^{2m}$, this lower bound is already sufficient to prove exponential growth with the polynomial degree. Therefore equispaced interpolation is not a uniformly stable interpolation process.

Chebyshev nodes. Now take the Chebyshev interpolation nodes

$$x_j = \cos\left(\frac{(2j+1)\pi}{2(n+1)}\right), \qquad j = 0, 1, \ldots, n.$$

Equivalently, writing $x = \cos\theta$, these nodes are equally spaced in the θ-variable:

$$\theta_j = \frac{(2j+1)\pi}{2(n+1)}, \qquad j = 0, 1, \ldots, n.$$

Under the change of variables $x = \cos\theta$, polynomial interpolation in x is transformed into interpolation by cosine polynomials in θ. Rivlin's analysis compares the corresponding cosine interpolation operator with Fourier partial sums and gives the estimate

$$\Lambda_n^{\text{cheb}} \leq \frac{2}{\pi} \log n + C_1,$$

where C_1 is independent of n [9]. Hence

$$\Lambda_n^{\text{cheb}} = O(\log n).$$

Moreover, the sharper asymptotic form is

$$\Lambda_n^{\text{cheb}} = \frac{2}{\pi} \log n + O(1), \qquad n \to \infty.$$

Thus Chebyshev interpolation has only logarithmic error amplification.

Rivlin also proves that for any triangular array of interpolation nodes X in $[-1,1]$, the corresponding Lebesgue constants satisfy

$$\Lambda_n(X) > \frac{2}{\pi^2}\log n - 1.$$

Therefore no sequence of interpolation nodes can make Λ_n remain bounded as n increases. Chebyshev nodes attain the same logarithmic order as this general lower bound. Hence Chebyshev nodes are order-optimal for stability: they do not remove the unavoidable growth of the Lebesgue constant, but they keep it at the smallest possible order.

2.3 Chebyshev nodes and Chebyshev basis (Task 1(c), Task 3(c), Task 4(b))

2.3.1 Chebyshev recurrence and orthogonality (Task 1(c))

Chebyshev polynomials of the first kind are defined by

$$T_n(x) = \cos(n \arccos x), \qquad x \in [-1,1]. \tag{1}$$

Let $x = \cos\theta$, then $T_n(x) = \cos(n\theta)$. The cosine addition identity gives

$$\cos((n+1)\theta) + \cos((n-1)\theta) = 2\cos\theta\cos(n\theta). \tag{2}$$

Therefore

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x), \tag{3}$$

with $T_0(x) = 1$ and $T_1(x) = x$.

For orthogonality, substitute $x = \cos\theta$. Since $dx = -\sin\theta\, d\theta$ and $\sqrt{1-x^2} = \sin\theta$ on $0 \le \theta \le \pi$,

$$\int_{-1}^{1} \frac{T_m(x)T_n(x)}{\sqrt{1-x^2}}\,dx = \int_0^{\pi} \cos(m\theta)\cos(n\theta)\,d\theta. \tag{4}$$

Thus

$$\int_{-1}^{1} \frac{T_m(x)T_n(x)}{\sqrt{1-x^2}}\,dx = \begin{cases} 0, & m \neq n, \\ \pi, & m = n = 0, \\ \pi/2, & m = n \geq 1. \end{cases} \tag{5}$$

The test function is

$$f(x) = e^x \sin(5x), \qquad x \in [-1,1]. \tag{6}$$

The training grid consists of $N = 200$ Chebyshev points. The mean squared error is measured on a separate test set of 1000 equally spaced points.

2.4 Least-squares approximation (Task 3(a), Task 3(b))

2.4.1 Monomial basis with normal equations (Task 3(b))

For degree n, the monomial Vandermonde matrix is

$$V_{ij} = x_i^j, \qquad i = 1,\ldots,N, \quad j = 0,\ldots,n. \tag{7}$$

The least-squares coefficients are computed using the normal equations

$$c = (V^TV)^{-1}V^Tf. \tag{8}$$

This approach is intentionally unstable because

$$\kappa_2(V^TV) = \kappa_2(V)^2. \tag{9}$$

Thus the ill-conditioning in V is squared by the normal equations.

2.5 QR method (Task 3(c))

2.5.1 Chebyshev basis with QR decomposition (Task 3(c))

Instead of using the monomial basis

$$1, x, x^2, \ldots, x^n,$$

we use the Chebyshev basis

$$T_0(x), T_1(x), \ldots, T_n(x).$$

The Chebyshev polynomials of the first kind are defined by

$$T_j(x) = \cos(j \arccos x), \qquad x \in [-1, 1].$$

The first few Chebyshev polynomials are

$$T_0(x) = 1, \qquad T_1(x) = x,$$

$$T_2(x) = 2x^2 - 1, \qquad T_3(x) = 4x^3 - 3x.$$

Equivalently, they satisfy the three-term recurrence

$$T_{j+1}(x) = 2xT_j(x) - T_{j-1}(x).$$

For the training points $x_1, \ldots, x_N$, the Chebyshev design matrix is

$$A^{(n)} = \begin{pmatrix} T_0(x_1) & T_1(x_1) & \cdots & T_n(x_1) \\ T_0(x_2) & T_1(x_2) & \cdots & T_n(x_2) \\ \vdots & \vdots & & \vdots \\ T_0(x_N) & T_1(x_N) & \cdots & T_n(x_N) \end{pmatrix}.$$

Equivalently,

$$A_{ij}^{(n)} = T_j(x_i), \qquad i = 1, \ldots, N, \qquad j = 0, \ldots, n.$$

The least-squares problem is

$$\min_{c \in \mathbb{R}^{n+1}} \|Ac - f\|_2.$$

Using the economy-size QR decomposition,

$$A = QR,$$

where $Q^TQ = I$ and R is upper triangular, the normal equations become

$$A^TAc = A^Tf.$$

Substituting $A = QR$, we get

$$(QR)^T(QR)c = (QR)^Tf.$$

Hence

$$R^TQ^TQRc = R^TQ^Tf.$$

Since $Q^TQ = I$, this simplifies to

$$R^TRc = R^TQ^Tf.$$

Assuming A has full column rank, R is nonsingular. Therefore R^T is also nonsingular, and we may cancel R^T, obtaining

$$Rc = Q^Tf.$$

Thus the least-squares coefficients are obtained by solving the triangular system

$$Rc = Q^T f.$$

Equivalently,

$$c = R^{-1} Q^T f.$$

In MATLAB, however, we do not explicitly form R^{-1}. Instead, we solve the triangular system using the backslash operator:

```
c_cheb = R_T \ (Q_T' * y_grid);.
```

2.5.2 Why the Chebyshev–QR method is more stable (Task 3(c))

The improved stability of the Chebyshev method comes from two separate effects: the use of a better-conditioned polynomial basis and the use of QR decomposition instead of normal equations.

For the monomial method, the design matrix is the Vandermonde matrix

$$V_{ij} = x_i^j, \qquad i = 1, \ldots, N, \qquad j = 0, \ldots, n.$$

The columns of V are

$$1,\ x,\ x^2,\ \ldots,\ x^n$$

evaluated on the grid. As n increases, these columns become increasingly close to linearly dependent on $[-1, 1]$. Hence the condition number

$$\kappa(V) = \frac{\sigma_{\max}(V)}{\sigma_{\min}(V)}$$

can become large.

The monomial method in this experiment solves the least-squares problem by the normal equations:

$$V^T V c = V^T f.$$

If the singular values of V are

$$\sigma_0(V) \geq \sigma_1(V) \geq \cdots \geq \sigma_n(V) > 0,$$

then the eigenvalues of $V^T V$ are

$$\sigma_0(V)^2,\ \sigma_1(V)^2,\ \ldots,\ \sigma_n(V)^2.$$

Therefore

$$\kappa(V^T V) = \frac{\sigma_0(V)^2}{\sigma_n(V)^2} = \left(\frac{\sigma_0(V)}{\sigma_n(V)}\right)^2 = \kappa(V)^2.$$

Thus the normal equations square the condition number. This means that even if V is only moderately ill-conditioned, the matrix $V^T V$ can be severely ill-conditioned.

For the Chebyshev method, the design matrix is instead

$$A_{ij} = T_j(x_i), \qquad i = 1, \ldots, N, \qquad j = 0, \ldots, n,$$

where T_j is the j-th Chebyshev polynomial. The Chebyshev polynomials are defined by

$$T_j(x) = \cos(j \arccos x), \qquad x \in [-1, 1].$$

They also satisfy

$$|T_j(x)| \leq 1, \qquad x \in [-1, 1].$$

Therefore the columns of the Chebyshev design matrix are much better scaled than the monomial columns $1, x, x^2, \ldots, x^n$.

In this coursework, the training points are Chebyshev points

$$x_i = \cos\left(\frac{(2i-1)\pi}{2N}\right), \qquad i = 1, \ldots, N.$$

Let

$$\theta_i = \frac{(2i-1)\pi}{2N}.$$

Then

$$A_{ij} = T_j(x_i) = \cos(j\theta_i).$$

For these nodes, the columns of A are nearly orthogonal. In fact, for $0 \le j, k \le n$ with $n < N$, the discrete orthogonality relation gives

$$\sum_{i=1}^{N} \cos(j\theta_i)\cos(k\theta_i) = \begin{cases} N, & j = k = 0, \\ \frac{N}{2}, & j = k > 0, \\ 0, & j \neq k. \end{cases}$$

Thus

$$A^T A = \operatorname{diag}\left(N, \frac{N}{2}, \frac{N}{2}, \ldots, \frac{N}{2}\right).$$

Hence the singular values of A are

$$\sqrt{N}, \quad \sqrt{\frac{N}{2}}, \quad \ldots, \quad \sqrt{\frac{N}{2}}.$$

Therefore

$$\kappa(A) = \frac{\sqrt{N}}{\sqrt{N/2}} = \sqrt{2} \approx 1.41.$$

This explains why the Chebyshev design matrix is very well-conditioned in the numerical experiment.

The important point is that QR decomposition does not form $A^T A$. Since Q has orthonormal columns, for any vector z,

$$\|Az\|_2 = \|QRz\|_2 = \|Rz\|_2.$$

Therefore A and R have the same singular values, and

$$\kappa(R) = \kappa(A).$$

Thus QR keeps the conditioning at the level of $\kappa(A)$, while the normal equations would lead to $\kappa(A^T A) = \kappa(A)^2$.

In summary, the Chebyshev–QR method is more stable than the monomial normal equation method because

$$\kappa(A_{\text{Cheb}}) \approx \sqrt{2},$$

whereas the monomial Vandermonde matrix has a rapidly growing condition number. Moreover, QR avoids the squaring of the condition number, while the monomial normal-equation method solves a system with condition number

$$\kappa(V^T V) = \kappa(V)^2.$$

2.6 Arnoldi orthonormal polynomial basis (Task 3(d))

2.6.1 Arnoldi-generated orthonormal basis by Gram–Schmidt (Task 3(d))

Algorithm 1 Arnoldi iteration for an orthonormal polynomial basis

Input: nodes $x = (x_1, \ldots, x_N)^T$, degree n
Output: orthonormal basis vectors $q_1, \ldots, q_{n+1}$, coefficients H
$q_1 = \mathbf{1}/\|\mathbf{1}\|_2$
for $j = 1, \ldots, n$ **do**
    $v = x \odot q_j$ ▷ multiply by x at each node
    for $i = 1, \ldots, j$ **do**
        $h_{ij} = q_i^T v$
        $v = v - h_{ij} q_i$
    end for
    $h_{j+1,j} = \|v\|_2$
    if $h_{j+1,j} = 0$ **then**
        break
    end if
    $q_{j+1} = v/h_{j+1,j}$
end for

The Arnoldi method constructs an orthonormal polynomial basis on the chosen training grid in the Algorithm 1. In this task, the training nodes are the $N = 200$ Chebyshev points

$$x_i = \cos\left(\frac{(2i-1)\pi}{2N}\right), \qquad i = 1, \ldots, N.$$

A polynomial is represented by its vector of values on these nodes:

$$q = \begin{pmatrix} q(x_1) \\ q(x_2) \\ \vdots \\ q(x_N) \end{pmatrix} \in \mathbb{R}^N.$$

The discrete inner product is

$$\langle u, v \rangle = u^T v = \sum_{k=1}^{N} u_k v_k.$$

Thus orthogonality is measured on the training grid, not by an exact continuous integral.

The multiplication operator M_x is defined by

$$(M_x q)(x) = x q(x).$$

On the discrete grid this is simply

$$M_x q = \mathrm{diag}(x_1, x_2, \ldots, x_N) q,$$

which is implemented in MATLAB by elementwise multiplication.

The key idea is that the monomial sequence

$$1,\ x,\ x^2, \ldots, x^n$$

can be generated by repeated multiplication by x. Instead of using these monomials directly, Arnoldi applies Gram–Schmidt orthogonalisation to the sequence generated by M_x. The starting vector is the constant polynomial sampled on the grid:

$$q_1 = \frac{1}{\sqrt{N}} \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}.$$

At step j, a new candidate vector is formed by

$$v = M_x q_j.$$

This vector is then orthogonalised against the previously constructed vectors $q_1, \ldots, q_j$. For $i = 1, \ldots, j$, define

$$h_{ij} = q_i^T v.$$

The projection of v in the direction q_i is $h_{ij} q_i$, so modified Gram–Schmidt updates

$$v \leftarrow v - h_{ij} q_i.$$

After all previous directions have been removed, set

$$h_{j+1,j} = \|v\|_2, \qquad q_{j+1} = \frac{v}{h_{j+1,j}}.$$

The orthogonality follows directly from Gram–Schmidt. After the projection subtractions at step j, the remaining vector is

$$u = M_x q_j - \sum_{i=1}^{j} h_{ij} q_i, \qquad h_{ij} = q_i^T M_x q_j.$$

For any $k = 1, \ldots, j$,

$$q_k^T u = q_k^T M_x q_j - \sum_{i=1}^{j} h_{ij} q_k^T q_i.$$

Since $q_1, \ldots, q_j$ are orthonormal,

$$q_k^T q_i = \begin{cases} 1, & k = i, \\ 0, & k \neq i. \end{cases}$$

Therefore

$$q_k^T u = q_k^T M_x q_j - h_{kj} = 0.$$

Hence u is orthogonal to all previous Arnoldi vectors, and normalising u gives a new unit vector q_{j+1}.

The Arnoldi recurrence is

$$M_x q_j = \sum_{i=1}^{j} h_{ij} q_i + h_{j+1,j} q_{j+1}.$$

After n steps, the matrix

$$Q = \begin{pmatrix} | & | & & | \\ q_1 & q_2 & \cdots & q_{n+1} \\ | & | & & | \end{pmatrix} \in \mathbb{R}^{N \times (n+1)}$$

satisfies

$$Q^T Q = I.$$

Thus the columns of Q form a discrete orthonormal polynomial basis on the training grid.

Least-Squares Approximation in the Arnoldi Basis

The corresponding least-squares problem in the Arnoldi basis is

$$\min_{c\in\mathbb{R}^{n+1}} \|Qc - f\|_2^2.$$

The normal equations are

$$Q^T Qc = Q^T f.$$

Since

$$Q^T Q = I,$$

we obtain immediately

$$c = Q^T f.$$

Hence the Arnoldi coefficients are simply projection coefficients:

$$c_j = q_j^T f, \qquad j = 0, 1, \ldots, n.$$

This is the main numerical advantage of the Arnoldi basis. No ill-conditioned Vandermonde system is solved. The least-squares matrix has orthonormal columns, so its condition number is

$$\kappa_2(Q) = 1.$$

The residual on the training grid is

$$r_n^{\text{Arnoldi}} = \frac{1}{\sqrt{N}}\|Qc - f\|_2.$$

The Hessenberg matrix H also provides a recurrence for evaluating the Arnoldi basis at new test points[10]. From

$$xq_j(x) = \sum_{i=0}^{j} h_{ij}q_i(x) + h_{j+1,j}q_{j+1}(x),$$

we solve for

$$q_{j+1}(x) = \frac{xq_j(x) - \sum_{i=0}^{j} h_{ij}q_i(x)}{h_{j+1,j}}.$$

Therefore, once $q_0(x)$ is known on a test grid, the remaining Arnoldi basis functions can be evaluated recursively using the stored Hessenberg coefficients.

2.7 Gauss–Legendre and Chebyshev–Gauss quadrature (Task 4(a), Task 4(b))

The quadrature component of Task 4 compares two polynomial expansion strategies for the endpoint-singular function

$$f(x) = \frac{e^{-x^2}}{\sqrt{1-x^2}}, \qquad x \in (-1, 1).$$

For the Legendre expansion, Gauss–Legendre nodes and weights approximate integrals with respect to the uniform weight. The coefficients are computed from

$$c_n = \frac{2n+1}{2}\int_{-1}^{1} f(x)P_n(x)\,dx.$$

For the Chebyshev expansion, Chebyshev–Gauss nodes

$$x_k = \cos\left(\frac{(2k-1)\pi}{2N}\right)$$

are used. Since $f(x)$ contains the factor $1/\sqrt{1-x^2}$, the Chebyshev calculation is regularised by approximating

$$g(x) = \sqrt{1-x^2}f(x) = e^{-x^2},$$

which is smooth on $[-1, 1]$. The final approximation restores the singular factor by dividing the polynomial approximation to g by $\sqrt{1-x^2}$.

3 Numerical Results and Analysis

3.1 Task 2: Runge phenomenon (Task 2(a)–(d))

The Runge function considered in this experiment is

$$f(x) = \frac{1}{1+25x^2}, \qquad x \in [-1, 1]. \tag{10}$$

Interpolation was carried out using $n+1$ equispaced nodes and $n+1$ Chebyshev nodes

$$x_k = \cos\left(\frac{(2k+1)\pi}{2(n+1)}\right), \qquad k = 0, \ldots, n. \tag{11}$$

The interpolants were evaluated using the Lagrange formula rather than solving a Vandermonde system directly.

Figure 1. Runge interpolation: equispaced versus Chebyshev nodes

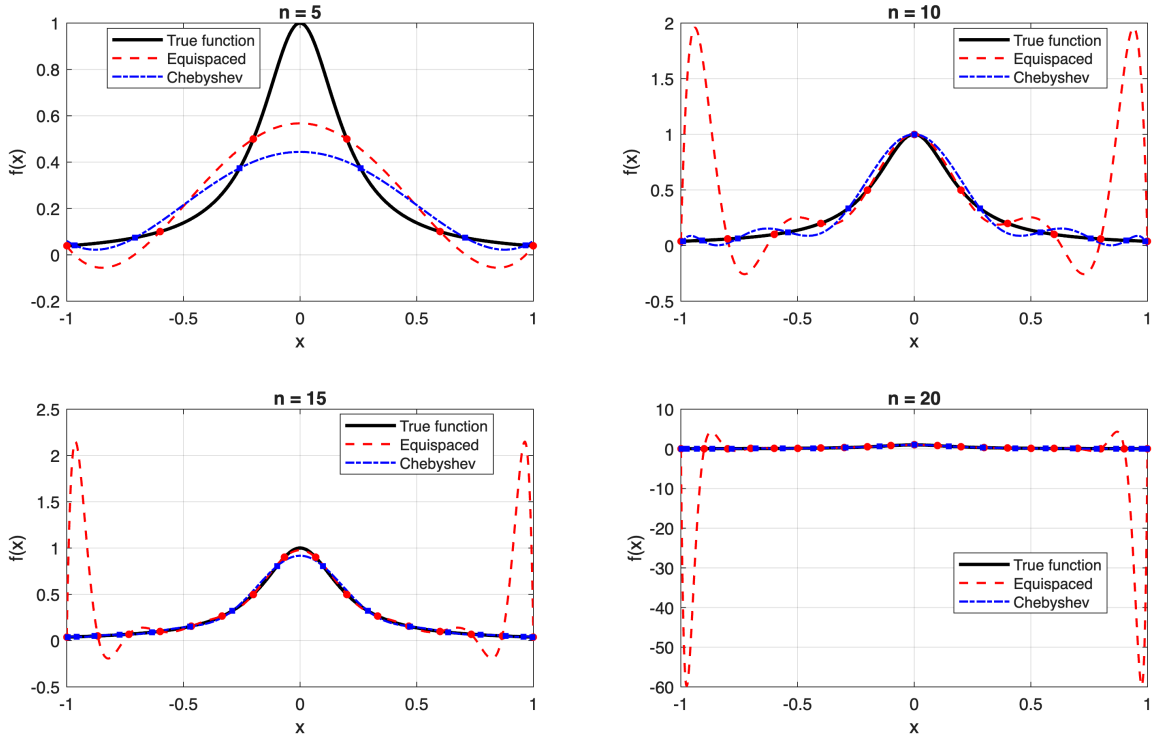


Figure 1: Runge interpolation using equispaced and Chebyshev nodes (CW2 Task 2(a), Task 2(b)) for $n = 5, 10, 15, 20$. As the degree increases, the equispaced interpolant develops large endpoint oscillations, while the Chebyshev interpolant remains close to the true function.

Figure 1 shows the classical Runge phenomenon. For equispaced nodes, the interpolating polynomial is accurate near the centre of the interval but develops increasingly large oscillations near the endpoints as n increases. In contrast, the Chebyshev interpolants remain much closer to the exact function over the whole interval.

This behaviour is explained by the Lebesgue constant analysis in Task 1(d). For interpolation nodes $x_0, \ldots, x_n$, the Lebesgue constant

$$\Lambda_n = \max_{x\in[-1,1]} \sum_{i=0}^{n} |\ell_i(x)|$$

measures the stability of the interpolation operator. The error estimate

$$\max_{x\in[-1,1]} |f(x) - I_n f(x)| \le (1 + \Lambda_n) \min_{p\in\mathcal{P}_n} \max_{x\in[-1,1]} |f(x) - p(x)|$$

shows that interpolation is close to best polynomial approximation only when Λ_n is small. For equispaced nodes, the Lebesgue constant grows essentially exponentially,

$$\Lambda_n^{\text{eq}} \sim \frac{2^{n+1}}{e\, n \log n},$$

whereas for Chebyshev nodes it grows only logarithmically,

$$\Lambda_n^{\text{cheb}} = \frac{2}{\pi} \log n + O(1).$$

The equispaced-node asymptotic explains the rapid amplification of interpolation error, while the logarithmic Chebyshev bound explains the much better stability of the Chebyshev interpolant [8, 9].

The Runge function also has complex singularities at

$$x = \pm\frac{i}{5}.$$

Although these singularities are not on the real interval, they are close to $[-1, 1]$. Hence high-degree interpolation is sensitive to the node distribution. Equispaced nodes do not control the endpoint growth of the Lagrange basis functions, so the interpolation error is strongly amplified near -1 and 1. Chebyshev nodes cluster near the endpoints, reduce the size of the Lagrange basis functions in these regions, and keep the Lebesgue constant near the best possible logarithmic order. In this precise stability sense, Chebyshev nodes minimise the Runge phenomenon.

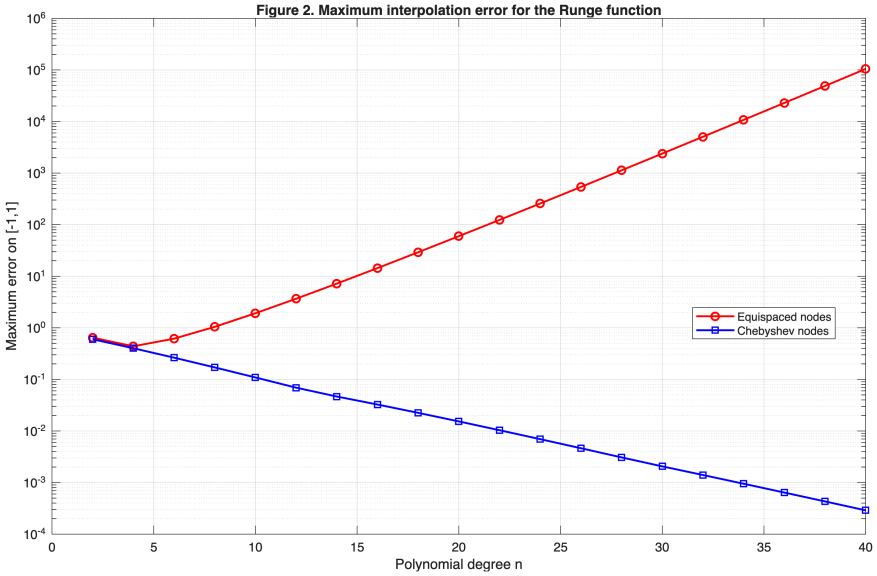


Figure 2: Maximum interpolation error (CW2 Task 2(b)) on $[-1, 1]$ as a function of the polynomial degree n. The vertical axis is logarithmic. The error for equispaced interpolation eventually grows rapidly, while the Chebyshev interpolation error decreases over the tested degree range.

Figure 2 confirms the theoretical prediction. The maximum error for equispaced interpolation initially decreases for small degrees but then increases rapidly as the endpoint oscillations dominate. The Chebyshev error, however, decreases over the tested degree range. This agrees with the fact that Chebyshev nodes have only logarithmic Lebesgue-constant growth.

At $n = 40$, the numerical experiment gives a maximum error of approximately 1.05×10^{5} for equispaced nodes but only 2.89×10^{-4} for Chebyshev nodes. The corresponding monomial Vandermonde condition numbers are approximately 2.97×10^{18} and 1.01×10^{15}, respectively. Thus, both the interpolation error and the conditioning results show that equispaced interpolation is substantially less stable.

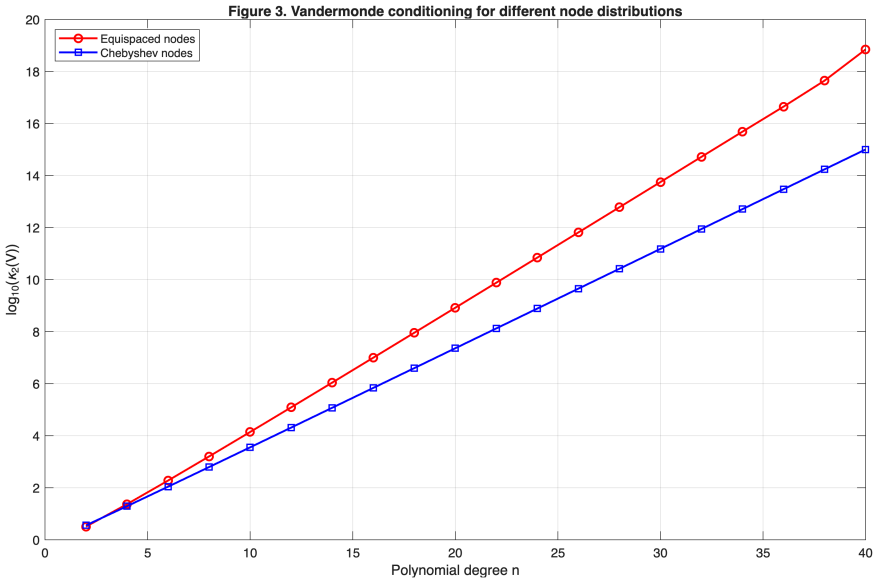


Figure 3: Conditioning of monomial Vandermonde matrices (CW2 Task 2(c)) for equispaced and Chebyshev nodes. The plotted quantity is $\log_{10}(\kappa_2(V))$. Both matrices become increasingly ill-conditioned as the degree increases, but the equispaced Vandermonde matrix has a significantly faster growth rate.

Figure 3 shows the growth of $\log_{10}(\kappa_2(V))$ for the monomial Vandermonde matrices. The condition number grows for both node choices, because the monomial basis itself is ill-conditioned for high-degree polynomial representation. Nevertheless, the equispaced Vandermonde matrix is consistently worse, confirming that node distribution and basis choice both affect numerical stability.

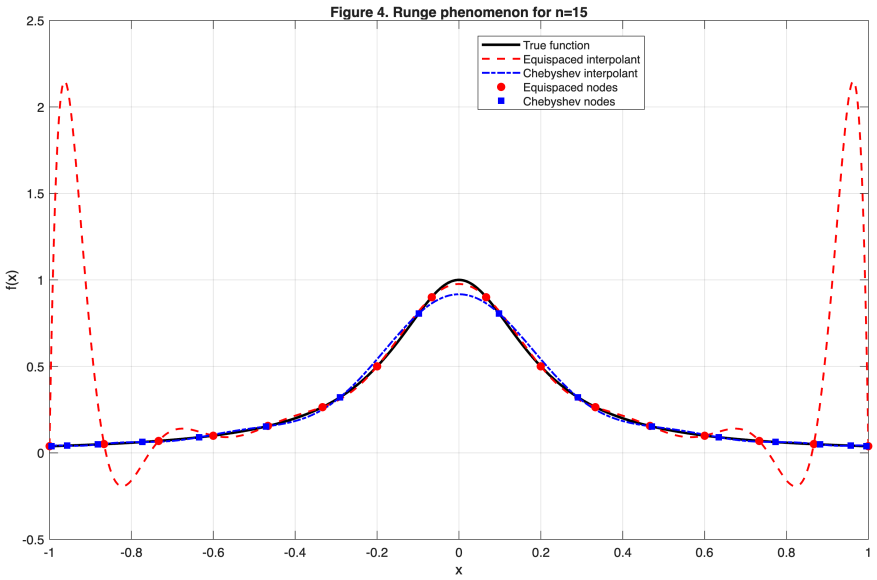


Figure 4: Runge phenomenon (CW2 Task 2(d)) for $n = 15$. The equispaced interpolant exhibits visible oscillations near the endpoints, whereas the Chebyshev interpolant follows the true Runge function much more closely. This figure isolates one representative degree to show the endpoint instability clearly.

Figure 4 isolates the case $n = 15$. The equispaced interpolant already exhibits clear endpoint oscillations, while the Chebyshev interpolant remains close to the Runge function. This single-degree comparison makes the practical effect of the Lebesgue constant visible: equispaced nodes produce large endpoint amplification, whereas Chebyshev nodes suppress it by placing more nodes near the endpoints.

3.2 Task 3: Monomial vs Chebyshev vs Arnoldi least squares (Task 3(a)–(e))

3.2.1 Arnoldi coefficient decay (Task 3(d))

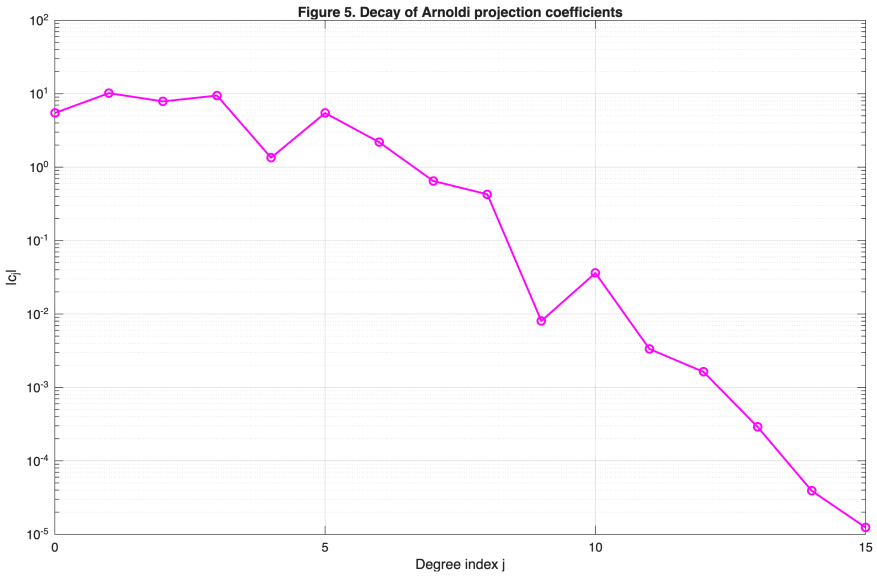


Figure 5: Decay of the Arnoldi projection coefficients (CW2 Task 3(d)) $|c_j|$ for the approximation of $f(x) = e^x \sin(5x)$. The coefficients are computed as $c_j = q_j^T f$ because the Arnoldi basis is orthonormal on the training grid. The overall decay indicates that the target function is well represented by a moderate-degree polynomial basis.

Figure 5 shows that the Arnoldi projection coefficients decrease rapidly as the polynomial degree increases. This coefficient decay is consistent with the smoothness of $f(x) = e^x \sin(5x)$ and indicates that only a moderate number of Arnoldi basis vectors is needed before the approximation reaches very small error on the training grid.

3.2.2 Error comparison (Task 3(e))

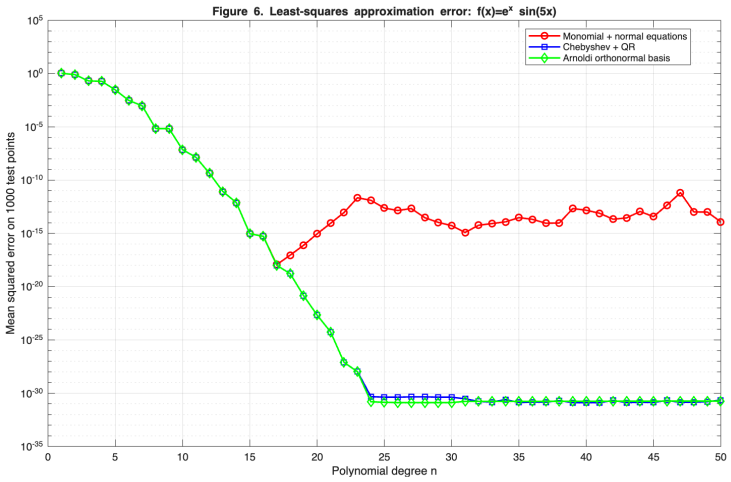


Figure 6: Mean squared error (CW2 Task 3(e)) on the independent test grid for the monomial normal-equation method, the Chebyshev QR method, and the Arnoldi orthonormal-basis method, shown for degrees up to $n = 50$. The Chebyshev QR and Arnoldi methods decrease to near machine precision and remain stable. The monomial method initially follows the same convergence trend but then stagnates and fluctuates because the normal equations are affected by the ill-conditioning of the Vandermonde matrix.

Figure 6 compares the actual approximation errors. For small and moderate degrees, the three curves are close because the target function is smooth and all three methods span the same polynomial space in exact arithmetic. The difference becomes visible at higher degrees: the Chebyshev QR and Arnoldi curves continue to decrease to near machine precision, while the monomial normal-equation curve stops improving and fluctuates. This behaviour agrees with the conditioning analysis below, since the monomial method uses a Vandermonde matrix and also squares its condition number through the normal equations.

3.2.3 Why the condition numbers differ (Task 3(e))

The three methods approximate the same function in the same polynomial space

$$\mathcal{P}_n.$$

Therefore, in exact arithmetic, they should give essentially the same least-squares polynomial. However, the numerical stability is very different because the design matrices have very different condition numbers.

1. Monomial basis with normal equations. For the monomial basis, the design matrix is the Vandermonde matrix

$$V_{ij} = x_i^j, \qquad i = 1, \ldots, N, \qquad j = 0, \ldots, n.$$

Its columns are the sampled values of

$$1,\ x,\ x^2, \ldots, x^n.$$

As n increases, these columns become increasingly close to linearly dependent on $[-1, 1]$. Therefore the smallest singular value of V becomes small, and the condition number

$$\kappa(V) = \frac{\sigma_{\max}(V)}{\sigma_{\min}(V)}$$

becomes large.

The monomial method solves the least-squares problem by the normal equations:

$$V^T V c = V^T f.$$

Let the singular values of V be

$$\sigma_0(V) \geq \sigma_1(V) \geq \cdots \geq \sigma_n(V) > 0.$$

Using the singular value decomposition

$$V = U \Sigma W^T,$$

we have

$$V^T V = W \Sigma^T U^T U \Sigma W^T = W \Sigma^T \Sigma W^T.$$

Since

$$\Sigma^T \Sigma = \operatorname{diag}\left(\sigma_0(V)^2, \sigma_1(V)^2, \ldots, \sigma_n(V)^2\right),$$

the singular values of $V^T V$ are the squared singular values of V. Hence

$$\kappa(V^T V) = \frac{\sigma_0(V)^2}{\sigma_n(V)^2} = \left(\frac{\sigma_0(V)}{\sigma_n(V)}\right)^2 = \kappa(V)^2.$$

Thus the normal equations square the condition number. This makes the monomial normal-equation method the least stable of the three approaches.

2. Chebyshev basis with QR decomposition. For the Chebyshev basis, the design matrix is

$$A_{ij} = T_j(x_i), \qquad i = 1, \ldots, N, \qquad j = 0, \ldots, n.$$

The Chebyshev polynomials satisfy

$$T_j(x) = \cos(j \arccos x),$$

and

$$|T_j(x)| \leq 1, \qquad x \in [-1, 1].$$

Thus the columns of the Chebyshev matrix are better scaled than the monomial columns.

In this experiment the training points are Chebyshev points

$$x_i = \cos\left(\frac{(2i-1)\pi}{2N}\right), \qquad i = 1, \ldots, N.$$

Let

$$\theta_i = \frac{(2i-1)\pi}{2N}.$$

Then

$$A_{ij} = T_j(x_i) = \cos(j\theta_i).$$

For $0 \le j, k \le n$ with $n < N$, the discrete orthogonality relation gives

$$\sum_{i=1}^{N} \cos(j\theta_i)\cos(k\theta_i) = \begin{cases} N, & j = k = 0, \\ \frac{N}{2}, & j = k > 0, \\ 0, & j \neq k. \end{cases}$$

Therefore

$$A^T A = \operatorname{diag}\left(N, \frac{N}{2}, \frac{N}{2}, \ldots, \frac{N}{2}\right).$$

Hence the singular values of A are

$$\sqrt{N}, \quad \sqrt{\frac{N}{2}}, \quad \ldots, \quad \sqrt{\frac{N}{2}}.$$

Thus

$$\kappa(A) = \frac{\sqrt{N}}{\sqrt{N/2}} = \sqrt{2}.$$

This explains why the Chebyshev design matrix is very well conditioned on the Chebyshev training grid.

The least-squares problem is solved using QR decomposition:

$$A = QR, \qquad Q^T Q = I.$$

For any vector z,

$$\|Az\|_2 = \|QRz\|_2 = \|Rz\|_2,$$

because multiplication by Q preserves Euclidean length. Therefore A and R have the same singular values, and

$$\kappa(R) = \kappa(A).$$

So QR does not square the condition number. It keeps the conditioning at the level of $\kappa(A)$, while normal equations would produce

$$\kappa(A^T A) = \kappa(A)^2.$$

3. Arnoldi-generated orthonormal basis. For the Arnoldi method, the design matrix is

$$Q = (q_0, q_1, \ldots, q_n).$$

By construction,

$$Q^T Q = I.$$

This means the columns of Q are orthonormal. Therefore, for any vector z,

$$\|Qz\|_2^2 = (Qz)^T (Qz) = z^T Q^T Q z = z^T z = \|z\|_2^2.$$

Hence

$$\|Qz\|_2 = \|z\|_2.$$

This shows that all singular values of Q are equal to 1. Therefore

$$\kappa(Q) = 1.$$

The least-squares coefficients are

$$c = Q^T f,$$

so no ill-conditioned system needs to be solved.

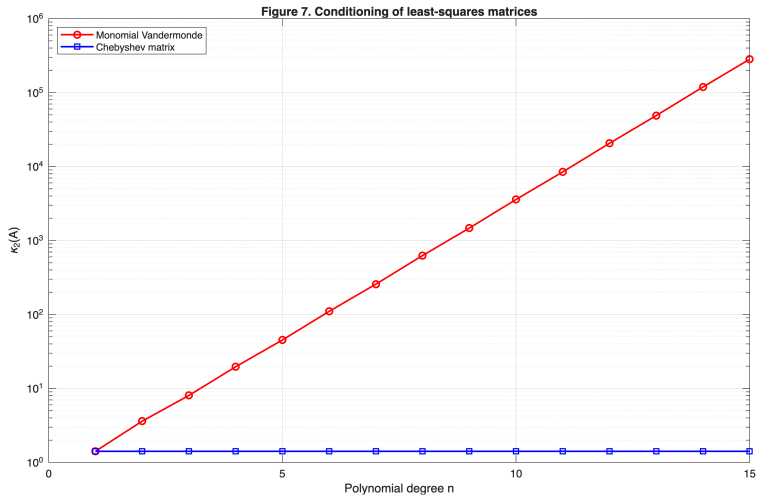


Figure 7: Condition numbers of the least-squares design matrices (CW2 Task 3(b), Task 3(c), Task 3(e)). The monomial Vandermonde matrix becomes rapidly ill-conditioned as the degree increases, while the Chebyshev design matrix remains close to a constant condition number on the Chebyshev training grid. This explains why the Chebyshev QR method is more stable than the monomial normal-equation method.

Figure 7 explains the behaviour in Figure 6. The monomial Vandermonde matrix becomes rapidly ill-conditioned as n increases, so solving the normal equations is numerically unreliable at high degree. The Chebyshev design matrix remains much better conditioned on the Chebyshev grid, and the Arnoldi basis is orthonormal by construction. Therefore the observed error curves are mainly determined by conditioning, not by the dimension of the polynomial space alone.

Summary. The three methods have different conditioning because they use different design matrices:

$$\text{Monomial: } V, \qquad \text{Chebyshev: } A, \qquad \text{Arnoldi: } Q.$$

The monomial Vandermonde matrix becomes ill-conditioned as n increases, and the normal equations square this ill-conditioning:

$$\kappa(V^T V) = \kappa(V)^2.$$

The Chebyshev matrix is much better conditioned because the Chebyshev basis is bounded and nearly orthogonal on the Chebyshev grid:

$$\kappa(A) \approx \sqrt{2}.$$

The Arnoldi matrix is best conditioned on the training grid because its columns are explicitly orthonormal:

$$\kappa(Q) = 1.$$

Therefore, the ranking in numerical stability is

$$\text{Arnoldi} \quad > \quad \text{Chebyshev with QR} \quad > \quad \text{Monomial with normal equations.}$$

The approximation errors may be similar for small or moderate n, because all three methods span the same polynomial space $\mathcal{P}_n$. The difference is not mainly the polynomial space, but the numerical conditioning of the basis and the solver.

3.2.4 Computational cost (Task 3(e))

Building a monomial Vandermonde matrix or Chebyshev matrix for N points and degree n costs $O(Nn)$. Solving by QR costs $O(Nn^2)$, while the normal-equation route forms V^TV in $O(Nn^2)$ and solves a small system in $O(n^3)$ but is less stable. Arnoldi with modified Gram-Schmidt also costs approximately $O(Nn^2)$. Its advantage is that the resulting basis is orthonormal on the training grid, so projection coefficients are obtained by inner products.

3.3 Task 4: Legendre vs regularised Chebyshev approximation (Task 4(a)–(d))

The endpoint-singular test function is

$$f(x) = \frac{e^{-x^2}}{\sqrt{1-x^2}}, \qquad x \in (-1,1). \tag{12}$$

This function has integrable endpoint singularities. Strictly speaking, $f \notin L^2(-1,1)$ because $f(x)^2 \sim (1-x^2)^{-1}$ has a logarithmic divergence at the endpoints. Therefore the numerical error reported here is a truncated interior L^2 error on $[-1+\delta, 1-\delta]$ with $\delta = 10^{-4}$.

3.3.1 Legendre approximation (Task 4(a))

Legendre polynomials satisfy

$$\int_{-1}^{1} P_m(x)P_n(x)\,dx = \frac{2}{2n+1}\delta_{mn}. \tag{13}$$

The Legendre coefficients are

$$c_n = \frac{2n+1}{2}\int_{-1}^{1} f(x)P_n(x)\,dx, \tag{14}$$

computed by Gauss-Legendre quadrature. Because the basis does not include the known endpoint singular factor, convergence is slow.

3.3.2 Regularised Chebyshev approximation (Task 4(b))

The Chebyshev weight is

$$w(x) = \frac{1}{\sqrt{1-x^2}}. \tag{15}$$

The singular factor in f suggests defining

$$g(x) = \sqrt{1-x^2}\,f(x) = e^{-x^2}. \tag{16}$$

The function g is smooth and analytic on $[-1,1]$. We approximate g using a Chebyshev expansion and reconstruct

$$f(x) \approx \frac{p_n(x)}{\sqrt{1-x^2}}. \tag{17}$$

This is the sense in which the Chebyshev weight cancels the endpoint singularity.

The test function in this task is

$$f(x) = \frac{e^{-x^2}}{\sqrt{1-x^2}}, \qquad x \in (-1,1). \tag{18}$$

This function has endpoint singularities at $x = \pm 1$, because

$$\sqrt{1-x^2} \to 0 \qquad \text{as} \qquad x \to \pm 1.$$

Hence

$$f(x) \to \infty \qquad \text{as} \qquad x \to \pm 1.$$

The singularities are integrable in the sense that

$$\int_{-1}^{1} \frac{1}{\sqrt{1-x^2}}\, dx < \infty.$$

However, the function is not square-integrable in the ordinary unweighted $L^2(-1,1)$ sense, because

$$f(x)^2 = \frac{e^{-2x^2}}{1-x^2} \sim \frac{C}{1-x^2} \qquad \text{near } x = \pm 1,$$

and

$$\int_{-1}^{1} \frac{1}{1-x^2}\, dx$$

diverges logarithmically. Therefore, the numerical L^2 error in this task is computed on the truncated interval

$$[-1+\delta, 1-\delta],$$

where

$$\delta = 10^{-4}.$$

This avoids evaluating the singular function exactly at the endpoints.

3.3.3 Legendre expansion (Task 4(a))

Legendre polynomials are orthogonal on $[-1,1]$ with respect to the uniform weight $w(x) = 1$. They satisfy

$$\int_{-1}^{1} P_m(x) P_n(x)\, dx = \frac{2}{2n+1} \delta_{mn}. \tag{19}$$

We approximate f by a truncated Legendre expansion

$$f_n^{\mathrm{Leg}}(x) = \sum_{k=0}^{n} c_k^{\mathrm{Leg}} P_k(x). \tag{20}$$

Using Legendre orthogonality, the expansion coefficients are

$$c_k^{\mathrm{Leg}} = \frac{2k+1}{2} \int_{-1}^{1} f(x) P_k(x)\, dx. \tag{21}$$

The integral is computed numerically by Gauss–Legendre quadrature. If

$$x_i^{\mathrm{L}}, \qquad w_i^{\mathrm{L}}, \qquad i = 1, \ldots, M_{\mathrm{L}},$$

are the Gauss–Legendre nodes and weights, then

$$\int_{-1}^{1} f(x) P_k(x)\, dx \approx \sum_{i=1}^{M_{\mathrm{L}}} w_i^{\mathrm{L}} f(x_i^{\mathrm{L}}) P_k(x_i^{\mathrm{L}}). \tag{22}$$

Therefore the numerical coefficient is

$$c_k^{\mathrm{Leg}} \approx \frac{2k+1}{2} \sum_{i=1}^{M_{\mathrm{L}}} w_i^{\mathrm{L}} f(x_i^{\mathrm{L}}) P_k(x_i^{\mathrm{L}}). \tag{23}$$

After computing the coefficients, the approximation is reconstructed by forming the finite sum

$$f_n^{\mathrm{Leg}}(x) = c_0^{\mathrm{Leg}} P_0(x) + c_1^{\mathrm{Leg}} P_1(x) + \cdots + c_n^{\mathrm{Leg}} P_n(x). \tag{24}$$

This reconstruction step is necessary because the coefficients alone are only the weights in the expansion. To compare with the true function and compute an error, we must evaluate the polynomial approximation itself.

In the MATLAB implementation, this is done by evaluating the matrix

$$P_{\mathrm{err}} = \begin{pmatrix} P_0(x_1) & P_1(x_1) & \cdots & P_n(x_1) \\ P_0(x_2) & P_1(x_2) & \cdots & P_n(x_2) \\ \vdots & \vdots & & \vdots \\ P_0(x_N) & P_1(x_N) & \cdots & P_n(x_N) \end{pmatrix}$$

on the error grid and computing

$$f_n^{\mathrm{Leg}} = P_{\mathrm{err}} c^{\mathrm{Leg}}. \tag{25}$$

The truncated L^2 error is then computed as

$$E_n^{\mathrm{Leg}} = \left(\int_{-1+\delta}^{1-\delta} \left| f(x) - f_n^{\mathrm{Leg}}(x) \right|^2 \, dx \right)^{1/2}. \tag{26}$$

Numerically this integral is evaluated using the trapezoidal rule on a fine grid.

The Legendre method approximates the singular function f directly. The basis functions $P_k(x)$ are ordinary polynomials and do not contain the known endpoint singular factor

$$\frac{1}{\sqrt{1-x^2}}.$$

Therefore many Legendre modes are needed to reproduce the endpoint behaviour, and the convergence is slow.

3.3.4 Regularised Chebyshev expansion (Task 4(b))

Chebyshev polynomials of the first kind are orthogonal with respect to the weight

$$w(x) = \frac{1}{\sqrt{1-x^2}}. \tag{27}$$

The standard Chebyshev coefficient formula for a smooth function u is

$$a_0 = \frac{1}{\pi} \int_{-1}^{1} \frac{u(x)}{\sqrt{1-x^2}} \, dx, \tag{28}$$

and, for $k \geq 1$,

$$a_k = \frac{2}{\pi} \int_{-1}^{1} \frac{u(x) T_k(x)}{\sqrt{1-x^2}} \, dx. \tag{29}$$

If this formula is applied directly to

$$f(x) = \frac{e^{-x^2}}{\sqrt{1-x^2}},$$

then the integrand contains

$$\frac{f(x)}{\sqrt{1-x^2}} = \frac{e^{-x^2}}{1-x^2},$$

which is not integrable at $x = \pm 1$. Hence the direct weighted Chebyshev coefficient formula is not appropriate for this singular function.

Instead, we use the known singular structure of f. Define the regularised function

$$g(x) = \sqrt{1 - x^2} f(x). \tag{30}$$

Substituting the definition of f, we obtain

$$g(x) = \sqrt{1 - x^2} \frac{e^{-x^2}}{\sqrt{1 - x^2}} = e^{-x^2}. \tag{31}$$

Thus the endpoint singularity has been removed. The function $g(x) = e^{-x^2}$ is smooth and analytic on $[-1, 1]$.

We now approximate g by a Chebyshev expansion:

$$g_n(x) = a_0 + \sum_{k=1}^{n} a_k T_k(x). \tag{32}$$

Using Chebyshev–Gauss nodes

$$x_j = \cos\left(\frac{(2j-1)\pi}{2M_{\mathrm{C}}}\right), \qquad j = 1, \ldots, M_{\mathrm{C}}, \tag{33}$$

and writing

$$\theta_j = \frac{(2j-1)\pi}{2M_{\mathrm{C}}}, \tag{34}$$

the coefficients are computed by the discrete cosine formulas

$$a_0 \approx \frac{1}{M_{\mathrm{C}}} \sum_{j=1}^{M_{\mathrm{C}}} g(x_j), \tag{35}$$

and, for $k \geq 1$,

$$a_k \approx \frac{2}{M_{\mathrm{C}}} \sum_{j=1}^{M_{\mathrm{C}}} g(x_j) \cos(k\theta_j). \tag{36}$$

Since

$$T_k(x_j) = \cos(k\theta_j),$$

these formulas are the Chebyshev–Gauss quadrature formulas in angular form.

After reconstructing g_n, we reconstruct the approximation to f by putting back the singular factor:

$$f_n^{\mathrm{Cheb}}(x) = \frac{g_n(x)}{\sqrt{1 - x^2}}. \tag{37}$$

This is the key difference from the Legendre method. The Legendre method tries to approximate the singular function f directly, whereas the Chebyshev regularised method approximates the smooth part $g = e^{-x^2}$ and then restores the known singular factor.

The truncated L^2 error is computed as

$$E_n^{\mathrm{Cheb}} = \left(\int_{-1+\delta}^{1-\delta} \left| f(x) - f_n^{\mathrm{Cheb}}(x) \right|^2 \, dx \right)^{1/2}. \tag{38}$$

3.3.5 Convergence comparison (Task 4(c))

The convergence rates are compared by plotting

$$E_n^{\text{Leg}} \qquad \text{and} \qquad E_n^{\text{Cheb}}$$

against the polynomial degree n on a semi-logarithmic scale.

The Legendre coefficients decay slowly because the Legendre basis does not include the endpoint singular factor $1/\sqrt{1-x^2}$. Therefore the Legendre series must use many polynomial terms to approximate the boundary singularity.

In contrast, the regularised Chebyshev method removes the singularity before constructing the polynomial approximation. It approximates

$$g(x) = e^{-x^2},$$

which is smooth and analytic on $[-1, 1]$. Smooth analytic functions have rapidly decaying Chebyshev coefficients, so the corresponding error decreases much faster.

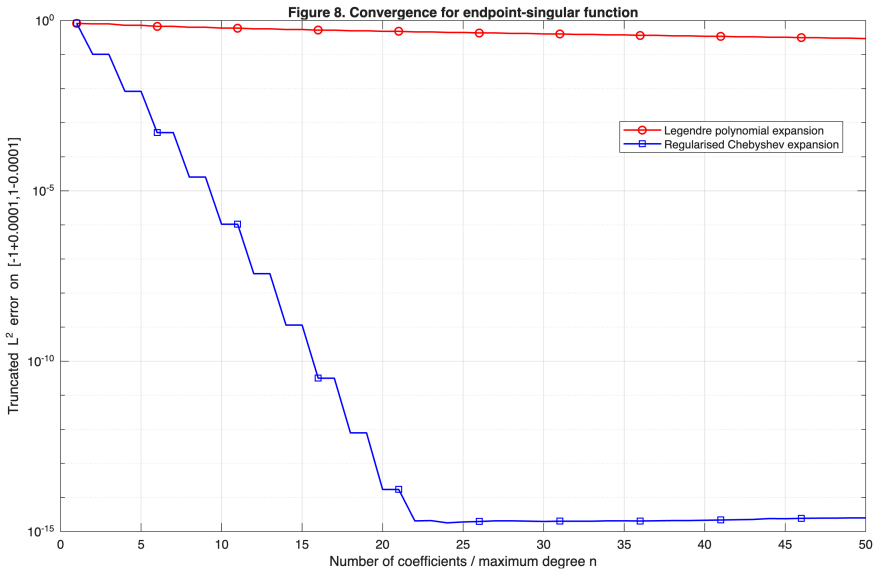


Figure 8: Truncated (CW2 Task 4(c)) L^2 error on $[-1+\delta, 1-\delta]$ with $\delta = 10^{-4}$ for the endpoint-singular function $f(x) = e^{-x^2}/\sqrt{1-x^2}$. The Legendre expansion approximates the singular function directly and converges slowly, whereas the regularised Chebyshev method first removes the singular factor and therefore reaches much smaller errors.

Figure 8 shows that the regularised Chebyshev approximation converges much faster than the direct Legendre expansion. The Legendre method approximates the endpoint-singular function itself, so the singularity slows the convergence. The Chebyshev method first removes the factor $1/\sqrt{1-x^2}$, approximates the smooth function $g(x) = e^{-x^2}$, and then restores the singular factor, which explains the much smaller error.

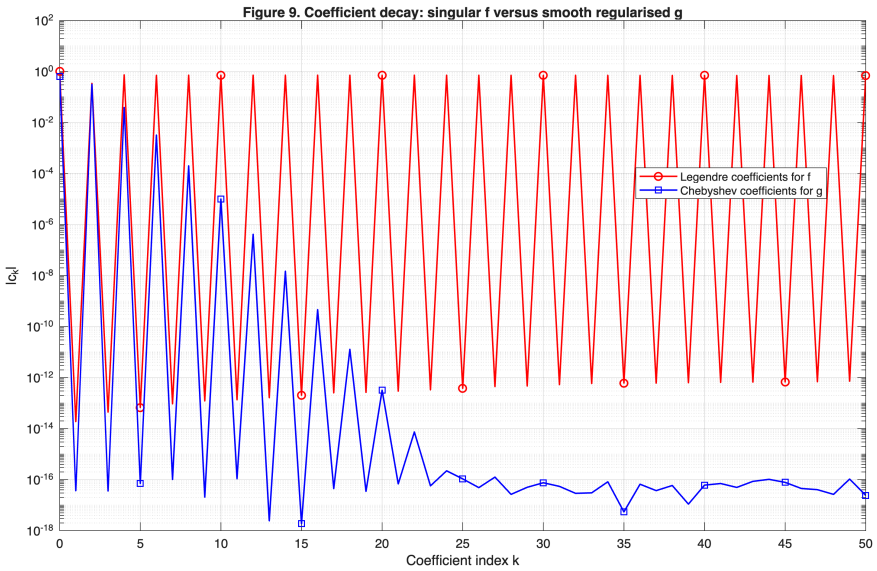


Figure 9: Coefficient decay (CW2 Task 4(c), Task 4(d)) for the Legendre expansion of the singular function f and the Chebyshev expansion of the regularised smooth function $g(x) = \sqrt{1-x^2}f(x) = e^{-x^2}$. The Chebyshev coefficients decay rapidly to near machine precision, while the Legendre coefficients decay much more slowly because the Legendre basis does not contain the endpoint singular factor.

Figure 9 supports the error comparison in Figure 8. The Chebyshev coefficients of the regularised smooth function decay rapidly, which indicates spectral-type convergence for $g(x) = e^{-x^2}$. In contrast, the Legendre coefficients for the original singular function decay slowly, reflecting the reduced smoothness caused by the endpoint singularities.

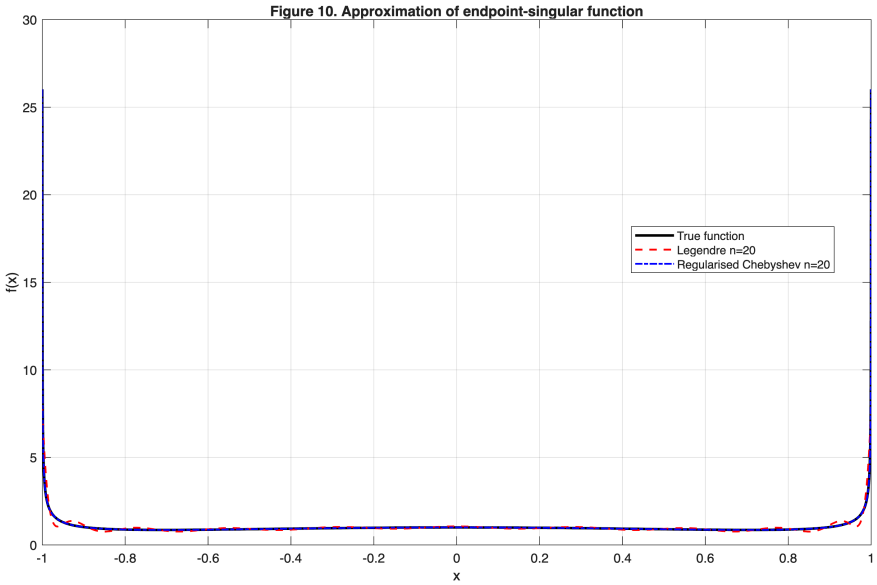


Figure 10: Approximation of the endpoint-singular function (CW2 Task 4(a), Task 4(b), Task 4(c)) $f(x) = e^{-x^2}/\sqrt{1-x^2}$ at degree $n = 20$. The regularised Chebyshev approximation closely follows the true function because it first approximates the smooth factor $g(x) = e^{-x^2}$ and then restores the singular factor $1/\sqrt{1-x^2}$. The Legendre approximation captures the general shape but is less accurate near the endpoints because it approximates the singular function directly.

Figure 10 gives a direct visual comparison at $n = 20$. Both approximations reproduce the

interior behaviour, but the regularised Chebyshev approximation better captures the endpoint growth because the known singular factor is built into the reconstruction.

For example, at degree $n = 20$, the truncated Legendre error is approximately 5.69×10^{-1}, while the regularized Chebyshev error is approximately 1.91×10^{-14}.

3.3.6 When Legendre approximation is preferable (Task 4(d))

Legendre expansion is preferable when the target function is naturally smooth with respect to the ordinary unweighted L^2 inner product

$$\langle u, v \rangle = \int_{-1}^{1} u(x)v(x)\,dx.$$

It is also appropriate when the physical error measure is genuinely unweighted, for example when all parts of the interval $[-1, 1]$ should contribute equally to the error.

Legendre expansion would be preferable when the target function is smooth in the ordinary uniform-weight L^2 setting or when the physical error measure is genuinely unweighted. Chebyshev-type methods are preferable when endpoint clustering, spectral accuracy, or known endpoint factors can be exploited.

For the present function, however, the dominant difficulty is the endpoint factor $1/\sqrt{1-x^2}$. Since this factor is known explicitly, the regularised Chebyshev method is more suitable.

3.3.7 Connection between Arnoldi and Gaussian quadrature (Task 4(d))

The Arnoldi method is related to Gaussian quadrature because applying Arnoldi or Lanczos to the multiplication operator generates recurrence coefficients for orthogonal polynomials. The eigenvalues of the resulting tridiagonal or Hessenberg recurrence matrix are Gaussian quadrature nodes, and the first components of the normalized eigenvectors determine quadrature weights.

There is also a connection between Arnoldi iteration and Gaussian quadrature. When Arnoldi or Lanczos iteration is applied to the multiplication operator

$$M_x q(x) = xq(x),$$

it generates recurrence coefficients for an orthonormal polynomial basis. These coefficients are stored in a Hessenberg or tridiagonal recurrence matrix.

The eigenvalues of this recurrence matrix correspond to Gaussian quadrature nodes for the associated inner product. The quadrature weights are obtained from the first components of the corresponding normalised eigenvectors. Therefore, Arnoldi provides a linear algebra construction of orthogonal polynomial recurrences and Gaussian quadrature rules.

4 Concluding Remarks

The numerical experiments support three conclusions. First, interpolation accuracy is not controlled by polynomial degree alone: node placement is crucial. Equispaced nodes give large Lebesgue constants and lead to the Runge phenomenon, while Chebyshev nodes greatly reduce endpoint oscillations. Second, the basis used for polynomial approximation matters in floating-point arithmetic. Monomial Vandermonde matrices become badly conditioned, and normal equations square the condition number. Chebyshev and Arnoldi bases remain much more stable. Third, functions with endpoint singularities should be approximated using a method that reflects the singular structure. Regularizing the singular factor before using a Chebyshev expansion gives much faster convergence than directly expanding the singular function in Legendre polynomials.

AI Use Acknowledgement

Thanks to AI assistance during the preparation of this coursework. .

In the written report, AI assistance was mainly used for parts of Task 1(b) and Task 1(d). For Task 1(b) I first derived the factorial form of the equispaced Vandermonde determinant independently, but then used AI assistance to help develop the remaining explanation connecting this determinant estimate to the growth of the condition number. For Task 1(d), I read the book introduced this proof [9] and then AI assistance was used to help formulate the Lebesgue-constant argument for explaining why Chebyshev nodes reduce the Runge phenomenon. These parts were then checked carefully, and AI was also used to revise mistakes identified during this checking process.

Then, the derivations in the methodology section were first written by myself in LaTeX, and AI was then used to help convert these methods into MATLAB code. The generated MATLAB code was reviewed and checked, and no errors were found during this review. After that, AI was also used to package the MATLAB code into separate function files with a main script.

AI assistance was also used for language polishing and organisation of the written discussion, especially in presenting the numerical results more clearly and improving the overall flow of the report. The mathematical interpretation and final judgement of the results were reviewed and confirmed.

The remaining parts of the report, including the overall interpretation, numerical analysis, and conclusions, were developed by me.

5 References

References

[1] Anthony Giunta and Layne Watson. A comparison of approximation modeling techniques–polynomial versus interpolating models. In *7th AIAA/USAF/NASA/ISSMO Symposium on Multidisciplinary Analysis and Optimization*, page 4758, 1998.

[2] Lloyd N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2019.

[3] Walter Gautschi. *Orthogonal Polynomials: Computation and Approximation*. Oxford University Press, 2004.

[4] John P. Boyd. *Chebyshev and Fourier Spectral Methods*. Dover, 2001.

[5] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4 edition, 2013.

[6] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. SIAM, 1997.

[7] Yousef Saad. *Numerical Methods for Large Eigenvalue Problems*. SIAM, 2011.

[8] Len Bos, Stefano De Marchi, Kai Hormann, and Georges Klein. On the lebesgue constant of barycentric rational interpolation at equidistant nodes. *Numerische Mathematik*, 121(3):461–471, 2012.

[9] Theodore J Rivlin. *An introduction to the approximation of functions*. Courier Corporation, 1981.

[10] Thomas Huckle. The arnoldi method for normal matrices. *SIAM Journal on Matrix Analysis and Applications*, 15(2):479–489, 1994.

A MATLAB Code

The full MATLAB script used to generate the figures and numerical summaries is listed below.

```
clear; close all; clc;
format short e;
rng(1);

figdir = '/Users/luyu/Desktop/cw2figures';

if ~exist(figdir, 'dir')
    mkdir(figdir);
end

%% ============================================================
% Task 2: Runge phenomenon
% ============================================================

runge = @(x) 1 ./ (1 + 25*x.^2);

n_vals = [5 10 15 20];
x_plot = linspace(-1,1,2000)';
f_true = runge(x_plot);

%% Figure 1: Runge interpolation for n = 5,10,15,20

fig1 = figure('Name','Task 2: Runge interpolants');

for k = 1:numel(n_vals)
    n = n_vals(k);

    % Equispaced nodes
    x_eq = linspace(-1,1,n+1)';
    y_eq = runge(x_eq);
    w_eq = bary_weights(x_eq);
    f_eq = bary_eval(x_plot,x_eq,y_eq,w_eq);

    % Chebyshev-Gauss nodes
    x_ch = cos((2*(0:n)' + 1)*pi/(2*(n+1)));
    y_ch = runge(x_ch);
    w_ch = bary_weights(x_ch);
    f_ch = bary_eval(x_plot,x_ch,y_ch,w_ch);

    subplot(2,2,k);
    plot(x_plot,f_true,'k-','LineWidth',2); hold on;
    plot(x_plot,f_eq,'r--','LineWidth',1.3);
    plot(x_plot,f_ch,'b-.','LineWidth',1.3);
    plot(x_eq,y_eq,'ro','MarkerSize',4,'MarkerFaceColor','r');
    plot(x_ch,y_ch,'bs','MarkerSize',4,'MarkerFaceColor','b');
    xlabel('x');
    ylabel('f(x)');
    title(sprintf('n = %d',n));
    legend('True function','Equispaced','Chebyshev','Location','best');
    grid on;
end

sgtitle('Figure 1. Runge interpolation: equispaced versus Chebyshev nodes');
save_png(fig1, figdir, 'fig1_runge_interpolants.png');


%% Figure 2 and Figure 3: max error and Vandermonde conditioning

n_err = 2:2:40;
err_eq = zeros(size(n_err));
err_ch = zeros(size(n_err));
cond_eq = zeros(size(n_err));
cond_ch = zeros(size(n_err));

for k = 1:numel(n_err)
    n = n_err(k);

    % Equispaced interpolation
    x_eq = linspace(-1,1,n+1)';
```

```
    y_eq = runge(x_eq);
    f_eq = bary_eval(x_plot,x_eq,y_eq,bary_weights(x_eq));
    err_eq(k) = max(abs(f_eq-f_true));

    V_eq = monomial_vandermonde(x_eq,n);
    cond_eq(k) = cond(V_eq);

    % Chebyshev interpolation
    x_ch = cos((2*(0:n)' + 1)*pi/(2*(n+1)));
    y_ch = runge(x_ch);
    f_ch = bary_eval(x_plot,x_ch,y_ch,bary_weights(x_ch));
    err_ch(k) = max(abs(f_ch-f_true));

    V_ch = monomial_vandermonde(x_ch,n);
    cond_ch(k) = cond(V_ch);
end

fig2 = figure('Name','Task 2: max interpolation error');
semilogy(n_err,err_eq,'r-o','LineWidth',1.5); hold on;
semilogy(n_err,err_ch,'b-s','LineWidth',1.5);
xlabel('Polynomial degree n');
ylabel('Maximum error on [-1,1]');
legend('Equispaced nodes','Chebyshev nodes','Location','best');
grid on;
title('Figure 2. Maximum interpolation error for the Runge function');
save_png(fig2, figdir, 'fig2_runge_error.png');

fig3 = figure('Name','Task 2: Vandermonde condition numbers');
plot(n_err,log10(cond_eq),'r-o','LineWidth',1.5); hold on;
plot(n_err,log10(cond_ch),'b-s','LineWidth',1.5);
xlabel('Polynomial degree n');
ylabel('log_{10}(\kappa_2(V))');
legend('Equispaced nodes','Chebyshev nodes','Location','northwest');
grid on;
title('Figure 3. Vandermonde conditioning for different node distributions');
save_png(fig3, figdir, 'fig3_vandermonde_condition.png');


%% Figure 4: Runge overlay for n = 15

n = 15;

x_eq = linspace(-1,1,n+1)';
y_eq = runge(x_eq);

x_ch = cos((2*(0:n)' + 1)*pi/(2*(n+1)));
y_ch = runge(x_ch);

f_eq = bary_eval(x_plot,x_eq,y_eq,bary_weights(x_eq));
f_ch = bary_eval(x_plot,x_ch,y_ch,bary_weights(x_ch));

fig4 = figure('Name','Task 2: Runge overlay n=15');
plot(x_plot,f_true,'k-','LineWidth',2); hold on;
plot(x_plot,f_eq,'r--','LineWidth',1.5);
plot(x_plot,f_ch,'b-.','LineWidth',1.5);
plot(x_eq,y_eq,'ro','MarkerFaceColor','r');
plot(x_ch,y_ch,'bs','MarkerFaceColor','b');
xlabel('x');
ylabel('f(x)');
legend('True function','Equispaced interpolant','Chebyshev interpolant', ...
       'Equispaced nodes','Chebyshev nodes','Location','best');
grid on;
title('Figure 4. Runge phenomenon for n=15');
save_png(fig4, figdir, 'fig4_runge_overlay_n15.png');


fprintf('\nTask 2 summary table:\n');
fprintf('  n       err_eq        err_ch        cond_eq       cond_ch\n');
for k = 1:numel(n_err)
    fprintf('%3d  %11.3e  %11.3e  %11.3e  %11.3e\n', ...
        n_err(k),err_eq(k),err_ch(k),cond_eq(k),cond_ch(k));
end

```

```
143
144 %% ============================================================
145 % Task 3: Least-squares approximation using Arnoldi
146 % ============================================================
147
148 f = @(x) exp(x).*sin(5*x);
149
150 N = 200;
151 x_grid = cos((2*(1:N)' - 1)*pi/(2*N));
152 y_grid = f(x_grid);
153
154 max_deg = 15;
155 N_test = 1000;
156 x_test = linspace(-1,1,N_test)';
157 y_test = f(x_test);
158
159 err_monomial  = zeros(max_deg,1);
160 err_chebyshev = zeros(max_deg,1);
161 err_arnoldi   = zeros(max_deg,1);
162
163 res_monomial  = zeros(max_deg,1);
164 res_chebyshev = zeros(max_deg,1);
165 res_arnoldi   = zeros(max_deg,1);
166
167 cond_monomial = zeros(max_deg,1);
168 cond_cheb     = zeros(max_deg,1);
169
170 arnoldi_coeff = [];
171
172 for n = 1:max_deg
173
174     % Monomial basis with normal equations
175     V = monomial_vandermonde(x_grid,n);
176     cond_monomial(n) = cond(V);
177
178     c_mono = (V'*V) \ (V'*y_grid);
179
180     V_test = monomial_vandermonde(x_test,n);
181     y_mono_test = V_test*c_mono;
182
183     err_monomial(n) = mean((y_mono_test-y_test).^2);
184     res_monomial(n) = norm(V*c_mono-y_grid)/sqrt(N);
185
186     % Chebyshev basis with QR
187     T = cheb_matrix(x_grid,n);
188     cond_cheb(n) = cond(T);
189
190     [Q_T,R_T] = qr(T,0);
191     c_cheb = R_T \ (Q_T'*y_grid);
192
193     T_test = cheb_matrix(x_test,n);
194     y_cheb_test = T_test*c_cheb;
195
196     err_chebyshev(n) = mean((y_cheb_test-y_test).^2);
197     res_chebyshev(n) = norm(T*c_cheb-y_grid)/sqrt(N);
198
199     % Arnoldi basis
200     [Q_A,H_A] = arnoldi_poly(x_grid,n);
201
202     c_arn = Q_A(:,1:n+1)'*y_grid;
203
204     Phi_test = arnoldi_eval(x_test,H_A,n,N);
205     y_arn_test = Phi_test*c_arn;
206
207     err_arnoldi(n) = mean((y_arn_test-y_test).^2);
208     res_arnoldi(n) = norm(Q_A(:,1:n+1)*c_arn-y_grid)/sqrt(N);
209
210     if n == max_deg
211         arnoldi_coeff = c_arn;
212     end
213 end
214
215
```

```
%% Figure 5: Arnoldi coefficient decay

fig5 = figure('Name','Task 3: Arnoldi coefficient decay');
semilogy(0:max_deg,abs(arnoldi_coeff),'m-o','LineWidth',1.5);
xlabel('Degree index j');
ylabel('|c_j|');
grid on;
title('Figure 5. Decay of Arnoldi projection coefficients');
save_png(fig5, figdir, 'fig5_arnoldi_coefficients.png');


%% Figure 6: least-squares error comparison

fig6 = figure('Name','Task 3: least-squares error comparison');
semilogy(1:max_deg,err_monomial,'r-o','LineWidth',1.5); hold on;
semilogy(1:max_deg,err_chebyshev,'b-s','LineWidth',1.5);
semilogy(1:max_deg,err_arnoldi,'g-d','LineWidth',1.5);
xlabel('Polynomial degree n');
ylabel('Mean squared error on 1000 test points');
legend('Monomial + normal equations','Chebyshev + QR','Arnoldi orthonormal basis', ...
       'Location','best');
grid on;
title('Figure 6. Least-squares approximation error: f(x)=e^x sin(5x)');
save_png(fig6, figdir, 'fig6_ls_errors.png');


%% Optional extra: residual errors

fig6b = figure('Name','Task 3: residual error comparison');
semilogy(1:max_deg,res_monomial,'r-o','LineWidth',1.5); hold on;
semilogy(1:max_deg,res_chebyshev,'b-s','LineWidth',1.5);
semilogy(1:max_deg,res_arnoldi,'g-d','LineWidth',1.5);
xlabel('Polynomial degree n');
ylabel('Residual error');
legend('Monomial + normal equations','Chebyshev + QR','Arnoldi orthonormal basis', ...
       'Location','best');
grid on;
title('Residual error versus polynomial degree');
save_png(fig6b, figdir, 'fig6b_residual_errors.png');


%% Figure 7: condition numbers

fig7 = figure('Name','Task 3: condition numbers');
semilogy(1:max_deg,cond_monomial,'r-o','LineWidth',1.5); hold on;
semilogy(1:max_deg,cond_cheb,'b-s','LineWidth',1.5);
xlabel('Polynomial degree n');
ylabel('\kappa_2(A)');
legend('Monomial Vandermonde','Chebyshev matrix','Location','northwest');
grid on;
title('Figure 7. Conditioning of least-squares matrices');
save_png(fig7, figdir, 'fig7_ls_condition.png');


fprintf('\nTask 3 summary table:\n');
fprintf(' n    mse_mono      mse_cheb      mse_arn       cond_mono     cond_cheb\n');
for n = 1:max_deg
    fprintf('%2d  %11.3e  %11.3e  %11.3e  %11.3e  %11.3e\n', ...
        n,err_monomial(n),err_chebyshev(n),err_arnoldi(n), ...
        cond_monomial(n),cond_cheb(n));
end


%% ============================================================
% Task 4: Legendre vs regularised Chebyshev approximation
% ============================================================

fsing = @(x) exp(-x.^2) ./ sqrt(1-x.^2);
greg  = @(x) exp(-x.^2);

n_max  = 50;
M_leg  = 800;
M_cheb = 1000;
```

```
delta  = 1e-4;
N_err  = 4000;

x_err = linspace(-1+delta,1-delta,N_err)';
y_err = fsing(x_err);

%% Legendre coefficients using Gauss-Legendre quadrature

[xL,wL] = gauss_legendre(M_leg);
fL = fsing(xL);

Pq = legendre_matrix(xL,n_max);

c_leg_all = zeros(n_max+1,1);

for k = 0:n_max
    c_leg_all(k+1) = (2*k+1)/2 * sum(wL .* fL .* Pq(:,k+1));
end

Perr = legendre_matrix(x_err,n_max);


%% Chebyshev coefficients for regularised function g(x)=exp(-x^2)

j = (1:M_cheb)';
theta = (2*j - 1)*pi/(2*M_cheb);
xC = cos(theta);

gC = greg(xC);

c_cheb_all = zeros(n_max+1,1);
c_cheb_all(1) = mean(gC);

for k = 1:n_max
    c_cheb_all(k+1) = (2/M_cheb) * sum(gC .* cos(k*theta));
end

Terr = cheb_matrix(x_err,n_max);


%% Reconstruction and error computation

error_legendre = zeros(n_max,1);
error_chebyshev_reg = zeros(n_max,1);

for n = 1:n_max

    % Legendre approximation to singular f
    y_leg = Perr(:,1:n+1)*c_leg_all(1:n+1);

    % Chebyshev approximation to smooth g, then reconstruct f
    g_cheb = Terr(:,1:n+1)*c_cheb_all(1:n+1);
    y_cheb_reg = g_cheb ./ sqrt(1-x_err.^2);

    % Truncated L2 error
    error_legendre(n) = sqrt(trapz(x_err,(y_leg-y_err).^2));
    error_chebyshev_reg(n) = sqrt(trapz(x_err,(y_cheb_reg-y_err).^2));
end


%% Figure 8: convergence comparison

fig8 = figure('Name','Task 4: Legendre vs regularised Chebyshev');
semilogy(1:n_max,error_legendre,'r-o', ...
    'LineWidth',1.2,'MarkerIndices',1:5:n_max);
hold on;
semilogy(1:n_max,error_chebyshev_reg,'b-s', ...
    'LineWidth',1.2,'MarkerIndices',1:5:n_max);
xlabel('Number of coefficients / maximum degree n');
ylabel(sprintf('Truncated L^2 error on [-1+%g,1-%g]',delta,delta));
legend('Legendre polynomial expansion','Regularised Chebyshev expansion', ...
       'Location','best');
grid on;
```

```
title('Figure 8. Convergence for endpoint-singular function');
save_png(fig8, figdir, 'fig8_legendre_chebyshev_errors.png');


%% Figure 9: coefficient decay

fig9 = figure('Name','Task 4: coefficient decay');
semilogy(0:n_max,abs(c_leg_all),'r-o', ...
    'LineWidth',1.2,'MarkerIndices',1:5:n_max+1);
hold on;
semilogy(0:n_max,abs(c_cheb_all),'b-s', ...
    'LineWidth',1.2,'MarkerIndices',1:5:n_max+1);
xlabel('Coefficient index k');
ylabel('|c_k|');
legend('Legendre coefficients for f','Chebyshev coefficients for g', ...
       'Location','best');
grid on;
title('Figure 9. Coefficient decay: singular f versus smooth regularised g');
save_png(fig9, figdir, 'fig9_coeff_decay.png');


%% Figure 10: approximation comparison for n = 20

n_show = 20;

y_leg_show = Perr(:,1:n_show+1)*c_leg_all(1:n_show+1);

g_cheb_show = Terr(:,1:n_show+1)*c_cheb_all(1:n_show+1);
y_cheb_show = g_cheb_show ./ sqrt(1-x_err.^2);

fig10 = figure('Name','Task 4: approximation comparison');
plot(x_err,y_err,'k-','LineWidth',2); hold on;
plot(x_err,y_leg_show,'r--','LineWidth',1.4);
plot(x_err,y_cheb_show,'b-.','LineWidth',1.4);
xlabel('x');
ylabel('f(x)');
legend('True function', ...
       sprintf('Legendre n=%d',n_show), ...
       sprintf('Regularised Chebyshev n=%d',n_show), ...
       'Location','best');
grid on;
title('Figure 10. Approximation of endpoint-singular function');
save_png(fig10, figdir, 'fig10_task4_approximation.png');


fprintf('\nTask 4 selected errors:\n');
fprintf(' n   Legendre truncated L2    Regularised Chebyshev truncated L2\n');

for n = [5 10 20 30 40 50]
    fprintf('%2d      %11.3e                 %11.3e\n', ...
        n,error_legendre(n),error_chebyshev_reg(n));
end


fprintf('\nAll figures saved to:\n%s\n', figdir);


%% ============================================================
% Local functions
% ============================================================

function save_png(fig, figdir, filename)
%SAVE_PNG Save a MATLAB figure as a high-resolution PNG.
    filepath = fullfile(figdir, filename);

    try
        exportgraphics(fig, filepath, 'Resolution', 300);
    catch
        saveas(fig, filepath);
    end

    fprintf('Saved: %s\n', filepath);
end
```

```
435
436
437 function V = monomial_vandermonde(x,n)
438 %MONOMIAL_VANDERMONDE  Vandermonde matrix [1 x x^2 ... x^n].
439     x = x(:);
440     V = ones(numel(x),n+1);
441
442     for j = 1:n
443         V(:,j+1) = V(:,j).*x;
444     end
445 end
446
447
448 function T = cheb_matrix(x,n)
449 %CHEB_MATRIX  Evaluate Chebyshev polynomials T_0,...,T_n.
450     x = x(:);
451     T = zeros(numel(x),n+1);
452
453     T(:,1) = 1;
454
455     if n >= 1
456         T(:,2) = x;
457     end
458
459     for k = 2:n
460         T(:,k+1) = 2*x.*T(:,k) - T(:,k-1);
461     end
462 end
463
464
465 function P = legendre_matrix(x,n)
466 %LEGENDRE_MATRIX  Evaluate Legendre polynomials P_0,...,P_n.
467     x = x(:);
468     P = zeros(numel(x),n+1);
469
470     P(:,1) = 1;
471
472     if n >= 1
473         P(:,2) = x;
474     end
475
476     for k = 1:n-1
477         P(:,k+2) = ((2*k+1)*x.*P(:,k+1) - k*P(:,k))/(k+1);
478     end
479 end
480
481
482 function w = bary_weights(x)
483 %BARY_WEIGHTS  Generic barycentric weights for interpolation nodes.
484     x = x(:);
485     m = numel(x);
486     w = ones(m,1);
487
488     for j = 1:m
489         diffj = x(j) - x([1:j-1,j+1:m]);
490         w(j) = 1/prod(diffj);
491     end
492
493     % Scaling does not change the barycentric interpolant.
494     w = w / max(abs(w));
495 end
496
497
498 function p = bary_eval(xeval,xnodes,ynodes,w)
499 %BARY_EVAL  Evaluate barycentric Lagrange interpolant.
500     xeval = xeval(:);
501     xnodes = xnodes(:);
502     ynodes = ynodes(:);
503     w = w(:);
504
505     p = zeros(numel(xeval),1);
506
507     for k = 1:numel(xeval)
```

```
        d = xeval(k) - xnodes;

        idx = find(abs(d) < 100*eps(max(1,abs(xeval(k)))),1);

        if ~isempty(idx)
            p(k) = ynodes(idx);
        else
            terms = w ./ d;
            p(k) = (terms.'*ynodes) / sum(terms);
        end
    end
end


function [Q,H] = arnoldi_poly(x,n)
%ARNOLDI_POLY  Arnoldi orthonormal polynomial basis on discrete nodes.
    x = x(:);
    N = numel(x);

    Q = zeros(N,n+1);
    H = zeros(n+1,n);

    Q(:,1) = ones(N,1)/sqrt(N);

    for j = 1:n
        v = x .* Q(:,j);

        % Modified Gram-Schmidt
        for i = 1:j
            H(i,j) = Q(:,i)'*v;
            v = v - H(i,j)*Q(:,i);
        end

        % Reorthogonalisation improves numerical orthogonality
        for i = 1:j
            hcorr = Q(:,i)'*v;
            H(i,j) = H(i,j) + hcorr;
            v = v - hcorr*Q(:,i);
        end

        H(j+1,j) = norm(v);

        if H(j+1,j) < 1e-14
            Q = Q(:,1:j);
            H = H(1:j,1:j-1);
            return;
        end

        Q(:,j+1) = v/H(j+1,j);
    end
end


function Phi = arnoldi_eval(x,H,n,Ntrain)
%ARNOLDI_EVAL  Evaluate Arnoldi basis using recurrence coefficients H.
    x = x(:);
    Phi = zeros(numel(x),n+1);

    Phi(:,1) = 1/sqrt(Ntrain);

    for j = 1:n
        v = x .* Phi(:,j);

        for i = 1:j
            v = v - H(i,j)*Phi(:,i);
        end

        Phi(:,j+1) = v / H(j+1,j);
    end
end


function [x,w] = gauss_legendre(n)
```

```
%GAUSS_LEGENDRE  Gauss-Legendre nodes and weights on [-1,1].
% Uses the Golub-Welsch eigenvalue method.
    if n == 1
        x = 0;
        w = 2;
        return;
    end

    k = (1:n-1)';
    beta = k ./ sqrt(4*k.^2 - 1);

    J = diag(beta,1) + diag(beta,-1);

    [V,D] = eig(J);
    x = diag(D);

    [x,idx] = sort(x);
    V = V(:,idx);

    w = 2*(V(1,:)').^2;
end
```